

# Full-Stack Developer

## Interview Preparation Guide

**MERN Stack & Next.js | All Levels — Junior to Senior**

8 Phases | 201 Questions | Rated by Interview Frequency

**Curated by N.I. Nazmul**

*Full-Stack Developer (MERN Stack & Next.js Specialist)*

*[github.com/ninazmul](https://github.com/ninazmul)*

### How to Read This Guide

|              |                     |                 |
|--------------|---------------------|-----------------|
| <b>BASIC</b> | <b>INTERMEDIATE</b> | <b>ADVANCED</b> |
|--------------|---------------------|-----------------|

★★★★★ = Asked Almost Every Interview | ★★★☆☆ = Commonly Asked | ★★☆☆☆ = Rarely Asked

|                                           |                                         |                                     |                                        |
|-------------------------------------------|-----------------------------------------|-------------------------------------|----------------------------------------|
| <b>Phase 1</b><br>JavaScript & TypeScript | <b>Phase 2</b><br>React & Next.js       | <b>Phase 3</b><br>Node.js & Express | <b>Phase 4</b><br>MongoDB & PostgreSQL |
| <b>Phase 5</b><br>System Design & DSA     | <b>Phase 6</b><br>Architecture & DevOps | <b>Phase 7</b><br>Testing & QA      | <b>Phase 8</b><br>API Design & GraphQL |

# Phase 1: JavaScript & TypeScript Fundamentals

32 Q&As

Q1

★★★★★

BASIC

**What is the difference between var, let, and const?**

**Answer:**

var is function-scoped and hoisted (initialized to undefined). let and const are block-scoped and live in the Temporal Dead Zone until declared. const cannot be reassigned, but its object/array contents can still be mutated.

```
var x = 1;
if (true) {
  var x = 10; // same variable – function-scoped
  let y = 20; // block-scoped, dies here
}
console.log(x); // 10
// console.log(y); // ReferenceError
```

*Interview tip: Prefer const by default; use let when reassignment is needed; avoid var in modern code.*

Q2

★★★★★

BASIC

**What is hoisting in JavaScript?**

**Answer:**

Hoisting moves variable and function declarations to the top of their scope before execution. var is hoisted and initialized to undefined. Function declarations are fully hoisted. let and const are hoisted but stay in the Temporal Dead Zone (TDZ) — accessing them before declaration throws a ReferenceError.

```
console.log(a); // undefined (var hoisted)
var a = 5;

greet(); // works – function fully hoisted
function greet() { return 'hello'; }

console.log(b); // ReferenceError (TDZ)
let b = 10;
```

*Interview tip: The TDZ is the gap between entering scope and the actual declaration line.*

Q3

★★★★★

BASIC

**Explain JavaScript closures with an example.**

**Answer:**

A closure is a function that remembers the variables from its birthplace (outer scope) even after that outer function has returned. Think of it as a backpack — the inner function carries its outer variables with it wherever it goes. Closures enable data encapsulation, factory functions, and stateful logic.

```
function makeCounter() {
  let count = 0;
  return {
    increment: () => ++count,
    decrement: () => --count,
    value: () => count,
  };
}
```

```
const c = makeCounter();
c.increment(); c.increment();
console.log(c.value()); // 2
```

*Interview tip: Each call to makeCounter() creates an independent closure with its own count.*

**Q4**

★★★★★

**BASIC**

## What is the difference between == and ===?

### Answer:

== (loose equality) coerces types before comparing — it tries to make both sides the same type. === (strict equality) compares value AND type with no coercion. Always use === in production code to avoid subtle type-coercion bugs.

```
0 == false      // true  (coercion)
0 === false     // false (different types)
null == undefined // true
null === undefined // false
'' == 0         // true
'' === 0        // false
```

*Interview tip: Use === everywhere. The only valid use of == is null-check: x == null catches both null and undefined.*

**Q5**

★★★★★

**BASIC**

## What are JavaScript primitive types?

### Answer:

Remember: SNBBUNS — String, Number, BigInt, Boolean, Undefined, Null, Symbol. These 7 are primitives — immutable and compared by value. Everything else (arrays, objects, functions, maps) is a reference type — compared by reference.

```
typeof 'hi'      // 'string'
typeof 42        // 'number'
typeof true      // 'boolean'
typeof undefined // 'undefined'
typeof null      // 'object' (JS bug)
typeof []        // 'object'
typeof function(){} // 'function'
```

*Interview tip: typeof null === 'object' is a historical JavaScript bug — use null === x to check for null.*

**Q6**

★★★★★

**BASIC**

## What is the difference between null and undefined?

### Answer:

undefined = 'nobody assigned a value yet' — it's the default empty state. null = 'I intentionally set this to nothing' — it's a deliberate absence. typeof undefined is 'undefined'; typeof null is 'object' (historical bug).

```
let a;           // undefined automatically
let b = null;    // intentional empty

function getUser(id) {
  if (!id) return null; // intentional: no user
}

console.log(a == b); // true (loose)
console.log(a === b); // false (strict)
```

*Interview tip: Use null when you intentionally want to represent 'no value'. Let undefined arise naturally.*

## Q7

★★★★★

BASIC

**Explain event delegation and event bubbling.****Answer:**

Event bubbling: when an event fires on an element, it bubbles UP through all its parents (child → parent → grandparent → document). Event delegation: instead of attaching listeners to every child, attach ONE listener to the parent and use event.target to identify which child was clicked. This is more memory-efficient and works for dynamically added elements.

```
// Instead of 1000 listeners on 1000 items:
ul.addEventListener('click', e => {
  if (e.target.tagName === 'LI') {
    console.log('Clicked:', e.target.textContent);
  }
});

// Phases: capture (down) → target → bubble (up)
// stopPropagation() stops the chain
// preventDefault() cancels default action
```

*Interview tip: Event delegation is how React handles events internally — one listener at the root.*

## Q8

★★★★★

BASIC

**What does the 'this' keyword refer to in different contexts?****Answer:**

this depends on HOW a function is called, not WHERE it's defined. Global: window/undefined. Object method: the object. Constructor (new): the new instance. Arrow function: inherits this from enclosing scope (lexical). call/apply/bind: whatever you pass. This is the #1 gotcha in JavaScript.

```
const obj = {
  name: 'Nazmul',
  greet() { return this.name; },      // 'Nazmul'
  arrow: () => this.name,              // undefined (lexical)
};

function show() { console.log(this); }
show();                               // window (or undefined in strict)
new show();                           // new object
show.call(obj);                       // obj
```

*Interview tip: Arrow functions don't have their own this — they capture it from where they're defined.*

## Q9

★★★★★

BASIC

**Explain destructuring, spread, and rest operators.****Answer:**

Destructuring extracts values from arrays/objects into variables. Spread (...) expands an iterable into individual elements. Rest (...) collects remaining elements into an array/object. Same syntax (...) but opposite directions — spread unpacks, rest packs.

```
// Destructuring
const { name, age = 25 } = user;
const [first, ...rest] = [1, 2, 3]; // rest = [2,3]

// Spread — clone + merge
const copy = { ...obj, newProp: 'val' };
const merged = [...arr1, ...arr2];

// Rest parameters
```

```
function sum(...nums) {
  return nums.reduce((a, b) => a + b, 0);
}
```

*Interview tip: Spread = unpack (array/object → elements). Rest = pack (elements → array/object).*

## Q10

★★★★★

BASIC

### What are optional chaining (?.) and nullish coalescing (??) operators?

#### Answer:

Optional chaining (?.) short-circuits to undefined if any part is null/undefined — no more 'cannot read property of undefined' errors. Nullish coalescing (??) returns the right side only when the left is null or undefined (not for 0, "", or false like || does).

```
// Optional chaining
const city = user?.address?.city; // safe
const first = arr?.[0];
const result = obj?.method?.();

// Nullish coalescing
const port = config.port ?? 3000;
// port = 0 → 0 (keeps falsy values!)
// port = null → 3000

// Compare with ||
const port2 = config.port || 3000;
// port = 0 → 3000 (BUG – treats 0 as falsy!)
```

*Interview tip: Use ?? when 0, "", and false are valid values. Use || only for truly falsy defaults.*

## Q11

★★★★★

BASIC

### What is the difference between Map and Object?

#### Answer:

Map allows any type as key (objects, functions, numbers); Object keys are always strings/symbols. Map maintains insertion order and has a .size property. Map is faster for frequent add/delete operations. Use Map for dynamic key-value lookups; Object for structured records.

```
const map = new Map();
map.set({id: 1}, 'user1'); // object as key!
map.set(42, 'answer');
map.size; // 2

// Iteration
for (const [key, val] of map) { ... }

// Object – keys coerced to strings
const obj = {};
obj[1] = 'one';
obj['1'] = 'ONE'; // overwrites! same key
Object.keys(obj); // ['1']
```

*Interview tip: Map for dictionaries with dynamic keys. Object for shaped data with known properties.*

## Q12



INTERMEDIATE

**Explain the JavaScript event loop.****Answer:**

JavaScript is single-threaded but non-blocking. The event loop is the traffic controller — it processes one task, then checks the microtask queue (Promises, queueMicrotask), drains it completely, then picks the next macrotask (setTimeout, setInterval). Microtasks always run before macrotasks.

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('4');
// Output: 1, 4, 3, 2
// Sync first → microtask (3) → macrotask (2)
```

*Interview tip: setTimeout(fn, 0) does not mean immediate — it means 'run after current + microtasks finish'.*

## Q13



INTERMEDIATE

**What is the difference between Promise, async/await, and callbacks?****Answer:**

Three generations of async: Callbacks (oldest, nesting hell) → Promises (.then/.catch chaining) → async/await (syntactic sugar, reads like sync code). async/await IS Promises underneath — just cleaner syntax. Always wrap await in try/catch for error handling.

```
// Callback hell
fetchUser(id, (err, user) => {
  fetchPosts(user.id, (err, posts) => {
    fetchComments(posts[0].id, (err, comments) => {});
  });
});

// async/await (clean)
async function load(id) {
  try {
    const user = await fetchUser(id);
    const posts = await fetchPosts(user.id);
    const comments = await fetchComments(posts[0].id);
  } catch(e) { console.error(e); }
}
```

*Interview tip: Always handle rejections. Unhandled promise rejections crash Node.js processes.*

## Q14



INTERMEDIATE

**Explain prototypal inheritance in JavaScript.****Answer:**

Every JS object has a hidden [[Prototype]] link to another object. When you access a property that doesn't exist on the object, JS walks up the prototype chain until it finds it or reaches null. ES6 classes are just syntactic sugar over this prototype chain mechanism.

```
function Animal(name) { this.name = name; }
Animal.prototype.speak = function() {
  return this.name + ' speaks';
};

const dog = new Animal('Rex');
console.log(dog.speak()); // Rex speaks
console.log(dog.__proto__ === Animal.prototype); // true
```

```
console.log(dog.hasOwnProperty('speak')); // false
```

*Interview tip: hasOwnProperty distinguishes own properties from inherited prototype properties.*

**Q15**

★★★★★

**INTERMEDIATE**

### Explain debounce vs throttle.

**Answer:**

Debounce: wait until the user STOPS doing something, then fire once. Like a search box — wait until they stop typing. Throttle: fire at most once per interval, no matter how many times triggered. Like scroll events — fire every 200ms max. Both prevent performance death from rapid events.

```
// Debounce – waits for silence
function debounce(fn, ms) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), ms);
  };
}

// Throttle – fires at intervals
function throttle(fn, ms) {
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= ms) {
      last = now;
      fn(...args);
    }
  };
}
```

*Interview tip: Debounce for search inputs. Throttle for scroll/resize. Both are critical for performance.*

**Q16**

★★★★★

**INTERMEDIATE**

### Explain the classic var + closure loop problem.

**Answer:**

With var in a for loop, all callbacks share the same variable reference because var is function-scoped. By the time callbacks execute, i holds its final value. Fix: use let (block-scoped per iteration) or an IIFE to capture each value.

```
// Problem – prints 3,3,3
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}

// Fix 1: let (block-scoped)
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0); // 0,1,2
}

// Fix 2: IIFE
for (var i = 0; i < 3; i++) {
  ((j) => setTimeout(() => console.log(j), 0))(i);
}
```

*Interview tip: This is one of the most common closure interview questions — know both fixes.*

## Q17



INTERMEDIATE

**What is the difference between call, apply, and bind?****Answer:**

All three let you choose what 'this' is. call() and apply() invoke the function immediately — call takes args as a list, apply takes an array (A for Array). bind() returns a NEW function with 'this' permanently locked — it doesn't invoke immediately.

```
function greet(greeting, punct) {
  return greeting + ', ' + this.name + punct;
}
const user = { name: 'Nazmul' };

greet.call(user, 'Hello', '!'); // 'Hello, Nazmul!'
greet.apply(user, ['Hi', '?']); // 'Hi, Nazmul?'
const fn = greet.bind(user, 'Hey');
fn('.'); // 'Hey, Nazmul.'
```

*Interview tip: bind is commonly used to preserve this in class methods passed as React event handlers.*

## Q18



INTERMEDIATE

**What are Promise.all, Promise.race, Promise.allSettled, and Promise.any?****Answer:**

Promise.all: ALL must succeed — one failure rejects everything. Promise.race: first to settle (success or fail) wins. Promise.allSettled: waits for ALL and never rejects — gives you status of each. Promise.any: first SUCCESS wins — only rejects if ALL fail (AggregateError).

```
const p1 = fetch('/api/users');
const p2 = fetch('/api/posts');

// All must succeed
const [users, posts] = await Promise.all([p1, p2]);

// Results regardless of failure
const results = await Promise.allSettled([p1, p2]);
results.forEach(r => {
  if (r.status === 'fulfilled') console.log(r.value);
  else console.log(r.reason);
});
```

*Interview tip: Use Promise.allSettled when you need results from all — even failed — promises.*

## Q19



INTERMEDIATE

**What are arrow functions and how do they differ from regular functions?****Answer:**

Arrow functions have 5 key differences: 1) No own 'this' (inherits from enclosing scope), 2) No 'arguments' object, 3) Cannot be used as constructors (no new), 4) No prototype property, 5) Concise syntax. They're perfect for callbacks but wrong for object methods.

```
// Regular function — has own 'this'
function Timer() {
  this.count = 0;
  setInterval(function() {
    this.count++; // 'this' is undefined in strict
  }, 1000);
}
```

```
// Arrow function – inherits 'this'
function Timer() {
  this.count = 0;
  setInterval(() => {
    this.count++; // 'this' is Timer instance
  }, 1000);
}
```

*Interview tip: Never use arrow functions as object methods if you need this to refer to the object.*

**Q20**

★★★★★

**INTERMEDIATE**

## Explain JavaScript modules: ESM vs CommonJS.

### Answer:

CommonJS (require/module.exports): synchronous, runs at runtime, used in Node.js. ESM (import/export): asynchronous, statically analyzed at compile time, supports tree-shaking. ESM is the future — Node.js 14+ supports it, browsers support it natively.

```
// CommonJS (Node.js traditional)
const fs = require('fs');
module.exports = { myFunc };

// ESM (modern standard)
import fs from 'fs';
export const myFunc = () => {};
export default class MyClass {}

// Key differences:
// ESM: static imports → tree-shakeable
// CJS: dynamic require() → can be conditional
// ESM: top-level await supported
// CJS: require is synchronous
```

*Interview tip: Use ESM for new projects. Set 'type': 'module' in package.json to enable ESM in Node.js.*

**Q21**

★★★★★

**INTERMEDIATE**

## What is the difference between Map/Set and WeakMap/WeakSet?

### Answer:

Map/Set hold strong references — keys stay in memory forever. WeakMap/WeakSet hold weak references — keys can be garbage collected when no other references exist. WeakMap/WeakSet are not iterable and have no .size. Use them when you need to attach metadata to objects without preventing GC.

```
const cache = new WeakMap();

function process(obj) {
  if (cache.has(obj)) return cache.get(obj);
  const result = heavyComputation(obj);
  cache.set(obj, result); // auto-removed when obj is GC'd
  return result;
}

// Regular Map would prevent GC of obj
// WeakMap allows GC – no memory leak

// Set vs WeakSet
const visited = new WeakSet();
visited.add(node); // auto-cleaned when node is GC'd
```

*Interview tip: WeakMap is the correct tool for associating data with DOM nodes without memory leaks.*

### What is the difference between shallow copy and deep copy?

**Answer:**

Shallow copy duplicates the top level but nested objects share references — changing nested data in the copy changes the original. Deep copy recursively clones everything — fully independent. Spread and Object.assign are shallow. structuredClone() is the modern deep copy.

```
const orig = { a: 1, b: { c: 2 } };

// Shallow – b is shared
const shallow = { ...orig };
shallow.b.c = 99;
console.log(orig.b.c); // 99 (mutated!)

// Deep – fully independent
const deep = structuredClone(orig);
deep.b.c = 99;
console.log(orig.b.c); // 2 (safe)
```

*Interview tip: JSON.parse(JSON.stringify()) fails on Date, undefined, functions, and circular refs — use structuredClone.*

### Explain TypeScript generics.

**Answer:**

Generics let you write functions/classes that work with ANY type while keeping type safety. Think of as a type variable — it captures the type at call time and carries it through. Without generics you'd use 'any' and lose all type information.

```
function identity<T>(arg: T): T { return arg; }

// Real-world: typed API response
async function fetchData<T>(url: string): Promise<T> {
  const res = await fetch(url);
  return res.json() as T;
}

interface User { id: number; name: string; }
const user = await fetchData<User>('/api/user/1');
user.name; // TypeScript knows this is string
```

*Interview tip: Use constraints (T extends object) to limit which types can be passed.*

### What are TypeScript utility types?

**Answer:**

Built-in type transformers. Partial makes all properties optional. Required makes all mandatory. Pick selects specific keys. Omit removes keys. Record maps keys to values. Readonly prevents mutation. These compose like building blocks.

```
interface User {
  id: number; name: string;
  email: string; age: number;
}

type UpdateUser = Partial<User>;
// All fields optional – for PATCH requests
```

```

type UserPreview = Pick<User, 'id' | 'name'>;
// Only id and name

type PublicUser = Omit<User, 'email'>;
// Everything except email

type UserMap = Record<string, User>;
// { [key: string]: User }

```

*Interview tip: Combine utility types: `Partial` for partial update DTOs.*

**Q25**

★★★★★

**ADVANCED**

## What is the difference between interface and type in TypeScript?

### Answer:

Both define shapes, but: Interfaces support 'extends' and declaration merging (same name merges automatically). Types handle unions, intersections, tuples, and mapped types. Rule of thumb: interface for object shapes, type for everything else.

```

interface Animal { name: string; }
interface Dog extends Animal { breed: string; }

// Declaration merging (interfaces only)
interface Window { myProp: string; }
interface Window { another: number; } // merged

// Types handle unions
type ID = string | number;
type Result<T> = { data: T } | { error: string };
type Point = [number, number]; // tuple

```

*Interview tip: In a codebase, pick one and be consistent. Most teams use interface for objects, type for the rest.*

**Q26**

★★★★★

**ADVANCED**

## What is TypeScript's `unknown` vs `any`?

### Answer:

`any` disables type checking — anything goes, no safety net. `unknown` is the type-safe counterpart — you MUST narrow the type before using it. Think: `any` = 'I don't care about types'. `unknown` = 'I don't know the type YET but I'll check before using it'.

```

// any – no safety (avoid!)
let x: any = getData();
x.foo.bar; // no error, might crash

// unknown – must check first (safe)
let y: unknown = getData();
// y.foo; // ERROR: Object is of type 'unknown'

if (typeof y === 'string') {
  y.toUpperCase(); // OK – narrowed to string
}

if (y instanceof User) {
  y.name; // OK – narrowed to User
}

```

*Interview tip: Use `unknown` for values from external sources (APIs, user input). Never use `any` unless migrating JS.*

### Explain TypeScript discriminated unions.

**Answer:**

A pattern where a union of types shares a common literal property (the discriminant) that TypeScript uses to narrow the type. Think of it as a 'type tag' — switch on the tag and TypeScript knows exactly which shape you're dealing with. Essential for Redux actions, API responses, and state machines.

```
type Shape =
  | { kind: 'circle'; radius: number }
  | { kind: 'rect'; w: number; h: number }
  | { kind: 'triangle'; base: number; height: number };

function area(s: Shape): number {
  switch (s.kind) {
    case 'circle': return Math.PI * s.radius ** 2;
    case 'rect':   return s.w * s.h;
    case 'triangle': return 0.5 * s.base * s.height;
  }
}
```

*Interview tip: The 'kind' field is the discriminant. TypeScript auto-narrows inside each case branch.*

### What is the TypeScript satisfies operator?

**Answer:**

satisfies (TS 4.9+) validates that a value matches a type WITHOUT widening the type. Unlike 'as', it preserves the narrowest inferred type while still type-checking. It catches errors at the assignment site, not at the usage site.

```
type Color = 'red' | 'green' | 'blue';
type Theme = Record<string, Color | Color[]>;

// With 'satisfies' — keeps literal types
const palette = {
  primary: 'red',
  secondary: ['green', 'blue'],
} satisfies Theme;

palette.primary.toUpperCase(); // OK — knows it's string
palette.secondary.map(c => c); // OK — knows it's array

// With 'as Theme' — loses specificity
// palette.secondary.map() would error
```

*Interview tip: Use satisfies for config objects where you want validation AND narrow type inference.*

### Explain JavaScript memory management and garbage collection.

**Answer:**

JS uses automatic garbage collection with a mark-and-sweep algorithm. Starting from roots (global, stack), it marks all reachable objects and sweeps (frees) unreachable ones. Memory leaks happen when references are kept alive unintentionally — they're never 'unreachable' so GC can't clean them.

```
// LEAK: listener never removed
function setup() {
  const bigData = new Array(1e6).fill('x');
  document.addEventListener('click', () => {
```

```

    console.log(bigData.length); // bigData never GC'd
  });
}

```

```

// FIX: remove listener
const handler = () => console.log('clicked');
document.addEventListener('click', handler);
// Cleanup:
document.removeEventListener('click', handler);

```

*Interview tip: Common leak sources: forgotten listeners, closures over big data, global variables, detached DOM nodes.*

**Q30**

★★★★★

**ADVANCED**

## What is the JavaScript Proxy object?

### Answer:

Proxy wraps an object and intercepts fundamental operations (get, set, delete, has, apply). It's like a middleware for object access. Combined with Reflect (which provides the default behavior), you can build reactive systems, validation layers, and logging. Vue 3's reactivity system is built on Proxy.

```

const handler = {
  get(target, key) {
    console.log('Getting: ' + key);
    return Reflect.get(target, key);
  },
  set(target, key, value) {
    if (typeof value !== 'number')
      throw new TypeError('Must be number');
    return Reflect.set(target, key, value);
  }
};
const state = new Proxy({}, handler);
state.count = 5; // validated
state.count;    // logs 'Getting: count'

```

*Interview tip: Proxy is how Vue 3 implements reactivity — understanding it is a strong senior signal.*

**Q31**

★★★★★

**ADVANCED**

## Explain JavaScript generators and iterators.

### Answer:

Generator functions (function\*) produce iterators via yield. They pause execution at each yield and resume on next(). This enables lazy evaluation (compute values only when needed), infinite sequences, and custom iteration protocols.

```

function* range(start, end, step = 1) {
  for (let i = start; i < end; i += step) yield i;
}
for (const n of range(0, 10, 2)) {
  console.log(n); // 0, 2, 4, 6, 8
}

function* fibonacci() {
  let [a, b] = [0, 1];
  while (true) { yield a; [a, b] = [b, a+b]; }
}
const fib = fibonacci();
fib.next().value; // 0
fib.next().value; // 1

```

*Interview tip: Generators power async iteration (for await...of) and are the foundation of Redux-Saga.*

## Explain Symbol and the iterator protocol.

### Answer:

Symbol creates globally unique identifiers — even `Symbol('foo') !== Symbol('foo')`. Well-known symbols (`Symbol.iterator`, `Symbol.toPrimitive`) let you customize built-in behavior. The iterator protocol: any object with a `[Symbol.iterator]()` method returning `{next()}` is iterable with `for...of`.

```
// Custom iterable
class Range {
  constructor(start, end) {
    this.start = start; this.end = end;
  }
  [Symbol.iterator]() {
    let current = this.start;
    const end = this.end;
    return {
      next() {
        return current <= end
          ? { value: current++, done: false }
          : { done: true };
      }
    };
  }
}

for (const n of new Range(1, 5)) console.log(n);
```

*Interview tip: Making your classes iterable unlocks `for...of`, `spread`, `destructuring`, and `Array.from()`.*

## Q33



BASIC

**What is the Virtual DOM?****Answer:**

The Virtual DOM is a lightweight JavaScript copy of the real DOM. On every state change, React creates a new virtual tree, diffs it against the previous one (reconciliation), and updates **ONLY** the changed nodes in the real DOM. This batching makes updates efficient — like sending one package instead of many small ones.

```
// React handles diffing automatically:
function Counter() {
  const [n, setN] = useState(0);
  return (
    <button onClick={() => setN(c => c+1)}>
      Count: {n}
    </button>
  );
  // Only the text node updates, not the whole button
```

*Interview tip: React's diffing algorithm assumes same-type elements at the same position are the same component.*

## Q34



BASIC

**What is the difference between controlled and uncontrolled components?****Answer:**

Controlled: React state is the single source of truth — value is driven by state, every keystroke fires onChange. Uncontrolled: the DOM is the source of truth, accessed via refs. Controlled is preferred for validation, conditional logic, and real-time formatting.

```
// Controlled
function Controlled() {
  const [val, setVal] = useState('');
  return (
    <input
      value={val}
      onChange={e => setVal(e.target.value)}
    />
  );
}

// Uncontrolled
function Uncontrolled() {
  const ref = useRef();
  const submit = () => console.log(ref.current.value);
  return <input ref={ref} />;
}
```

*Interview tip: File inputs are always uncontrolled — their value is read-only in the DOM.*

## Q35

★★★★★

BASIC

**What is the difference between `useEffect` and `useLayoutEffect`?****Answer:**

`useEffect` runs AFTER the browser paints — async, non-blocking, preferred for most side effects. `useLayoutEffect` runs BEFORE the browser paints — synchronous, blocks rendering. Use `useLayoutEffect` only when you need to read/mutate layout to avoid visual flickering.

```
// useEffect – after paint (async, no flicker risk)
useEffect(() => {
  document.title = 'Updated: ' + count;
}, [count]);

// useLayoutEffect – before paint (sync)
useLayoutEffect(() => {
  const { height } = ref.current.getBoundingClientRect();
  setHeight(height); // set before user sees anything
}, []);
```

*Interview tip: Only reach for `useLayoutEffect` when you see visual flickering with `useEffect`.*

## Q36

★★★★★

BASIC

**What role do keys play in React lists?****Answer:**

Keys are React's identity system for list items. During reconciliation, React uses keys to match old and new elements. Bad keys (array index) cause state bugs when items reorder. Good keys (stable unique IDs) let React correctly track additions, removals, and reordering.

```
// Bad – using index as key
items.map((item, i) => <Item key={i} data={item} />);
// If items reorder, React thinks content
// changed in place – local state is wrong

// Good – stable unique ID
items.map(item => (
  <Item key={item.id} data={item} />
));
```

*Interview tip: Keys must be stable across renders, unique among siblings, and not randomly generated at render time.*

## Q37

★★★★★

INTERMEDIATE

**Explain `useMemo` and `useCallback` — when to use them.****Answer:**

`useMemo` memoizes a computed VALUE — recomputes only when deps change. `useCallback` memoizes a FUNCTION reference — creates new function only when deps change. Both exist to prevent expensive recalculations or child re-renders. But they have overhead too — don't use prematurely.

```
// useMemo – skip re-sorting on unrelated renders
const sorted = useMemo(
  () => [...items].sort((a,b) => a.price - b.price),
  [items]
);

// useCallback – stable ref for memo'd child
const handleClick = useCallback(
  () => dispatch({ type: 'INCREMENT' }),
  [dispatch]
```

```
);  
  
<ExpensiveChild onClick={handleClick} />
```

*Interview tip: useMemo/useCallback have overhead too. Add them only after profiling shows a real problem.*

**Q38**

★★★★★

**INTERMEDIATE**

## How does React Context API work? What are its limitations?

### Answer:

Context passes data through the component tree without prop drilling. Create → Provider → useContext. The big limitation: ANY change to the Provider value re-renders ALL consumers — even those not using the changed value. For high-frequency state, use Zustand/Redux instead.

```
const ThemeCtx = createContext('light');  
  
function App() {  
  const [theme, setTheme] = useState('light');  
  return (  
    <ThemeCtx.Provider value={{ theme, setTheme }}>  
      <Layout />  
    </ThemeCtx.Provider>  
  );  
}  
  
function Button() {  
  const { theme } = useContext(ThemeCtx);  
  return <button className={theme}>Click</button>;  
}
```

*Interview tip: Split context by concern — separate ThemeContext from UserContext to avoid unnecessary re-renders.*

**Q39**

★★★★★

**INTERMEDIATE**

## How do you prevent unnecessary re-renders?

### Answer:

6 techniques: 1) React.memo — skip re-render if props unchanged. 2) useCallback — stable function refs. 3) useMemo — memoize derived values. 4) State colocation — keep state close to where it's used. 5) Context splitting — separate fast from slow context. 6) Virtualize long lists (react-window).

```
// React.memo  
const Card = React.memo(({ user }) => (  
  <div>{user.name}</div>  
));  
  
// Colocate state – don't lift unnecessarily  
function Form() {  
  // This state only affects Form  
  const [value, setValue] = useState('');  
  return <input value={value} onChange={e => setValue(e.target.value)} />;  
}
```

*Interview tip: Use React DevTools Profiler to identify what's actually re-rendering before optimizing.*

**Explain Next.js rendering: SSR, SSG, ISR, CSR.****Answer:**

4 rendering strategies: SSR (Server-Side Rendering) — HTML generated per request, always fresh. SSG (Static Site Generation) — built at compile time, fastest. ISR (Incremental Static Regeneration) — SSG that auto-refreshes on a timer. CSR (Client-Side Rendering) — rendered in browser, used behind auth.

```
// SSG (force-cache = static)
async function Page() {
  const data = await fetch(url, { cache: 'force-cache' });
  return <Render data={data} />;
}

// ISR – revalidate every 60 seconds
async function Page() {
  const data = await fetch(url, { next: { revalidate: 60 } });
  return <Render data={data} />;
}

// SSR – always fresh
async function Page() {
  const data = await fetch(url, { cache: 'no-store' });
  return <Render data={data} />;
}
```

*Interview tip: ISR is the default recommendation for most marketing and content pages.*

**What is the difference between Server Components and Client Components?****Answer:**

Server Components (default in App Router) run on the server — zero JS sent to browser, can access DB directly, but no hooks or browser APIs. Client Components ('use client') run in the browser — full React with hooks and events, but add to JS bundle. Push 'use client' as low as possible.

```
'use client';
import { useState } from 'react';

// Client Component
export function Counter() {
  const [n, setN] = useState(0);
  return <button onClick={() => setN(n+1)}>{n}</button>;
}

// Server Component (no directive)
async function UserList() {
  const users = await db.user.findMany();
  return users.map(u => <li key={u.id}>{u.name}</li>);
}
```

*Interview tip: Server Components reduce JS bundle size. Push 'use client' as far down the tree as possible.*

## Explain Suspense and Error Boundaries.

### Answer:

Suspense shows fallback UI while async content loads (like a loading spinner). Error Boundaries catch JS errors in child components and show fallback UI instead of crashing the whole app. Together they create resilient UIs that handle loading and error states gracefully.

```
// Suspense – loading state
<Suspense fallback={<Spinner />}>
  <UserProfile /> {/* async component */}
</Suspense>

// Error Boundary (class component)
class ErrorBoundary extends React.Component {
  state = { hasError: false };
  static getDerivedStateFromError(error) {
    return { hasError: true };
  }
  render() {
    if (this.state.hasError)
      return <h2>Something went wrong.</h2>;
    return this.props.children;
  }
}
```

*Interview tip: Nest Suspense boundaries: coarse for layout, fine-grained for individual data sections.*

## How do you write custom hooks?

### Answer:

Custom hooks extract reusable stateful logic into a function starting with 'use'. They can call other hooks. They share logic, not state — each component using the hook gets its own state. This is the primary way to reuse logic in functional React.

```
function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const stored = localStorage.getItem(key);
    return stored ? JSON.parse(stored) : initialValue;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

// Usage
const [theme, setTheme] = useLocalStorage('theme', 'dark');
```

*Interview tip: Name custom hooks with 'use' prefix — React's linter relies on this to check hook rules.*

**Explain useReducer and when to prefer it over useState.****Answer:**

useReducer manages complex state via a pure reducer function (state + action → new state). Prefer it when: state has multiple interrelated sub-values, transitions are complex, or you want testable logic separate from the component. It's like a mini Redux inside your component.

```
type Action =
  | { type: 'INCREMENT' }
  | { type: 'DECREMENT' }
  | { type: 'RESET' };

function reducer(state: number, action: Action) {
  switch (action.type) {
    case 'INCREMENT': return state + 1;
    case 'DECREMENT': return state - 1;
    case 'RESET': return 0;
    default: return state;
  }
}

const [count, dispatch] = useReducer(reducer, 0);
dispatch({ type: 'INCREMENT' });
```

*Interview tip: useReducer + Context is a lightweight alternative to Redux for medium complexity.*

**Compare state management: Redux vs Zustand vs Jotai.****Answer:**

Redux: predictable, verbose, mature ecosystem, best for large teams with complex state. Zustand: simple API (no boilerplate), uses hooks, great for medium apps. Jotai: atom-based (bottom-up), minimal re-renders, great for derived/async state. Pick based on team size and complexity.

```
// Zustand – simplest API
const useStore = create((set) => ({
  count: 0,
  inc: () => set((s) => ({ count: s.count + 1 })),
}));

function Counter() {
  const count = useStore((s) => s.count);
  const inc = useStore((s) => s.inc);
  return <button onClick={inc}>{count}</button>;
}

// Jotai – atom-based
const countAtom = atom(0);
const [count, setCount] = useAtom(countAtom);
```

*Interview tip: Start with useState → useReducer → Zustand → Redux. Don't over-engineer state management.*

### Explain useTransition and useDeferredValue.

**Answer:**

Both handle non-urgent updates without blocking the UI. useTransition: wraps a state update as low-priority — shows old content while computing new. useDeferredValue: creates a deferred copy of a value that updates on a delay. Both keep the UI responsive during expensive renders.

```
// useTransition – non-blocking state update
function Search() {
  const [query, setQuery] = useState('');
  const [isPending, startTransition] = useTransition();
  const [results, setResults] = useState([]);

  function handleChange(e) {
    setQuery(e.target.value); // urgent
    startTransition(() => {
      setResults(filterData(e.target.value)); // deferred
    });
  }
  return <{isPending && <Spinner />}</>;
}
```

*Interview tip: useTransition for state updates you control. useDeferredValue for values from props/parent.*

### What are React Server Actions?

**Answer:**

Server Actions (React 19 / Next.js 14+) let you define server-side functions that can be called directly from Client Components. They replace API routes for form submissions and mutations. Mark with 'use server' and use with form action={} or call programmatically.

```
// app/actions.ts
'use server';
export async function createPost(formData: FormData) {
  const title = formData.get('title');
  await db.post.create({ data: { title } });
  revalidatePath('/posts');
}

// app/page.tsx
import { createPost } from './actions';
export default function Page() {
  return (
    <form action={createPost}>
      <input name='title' />
      <button type='submit'>Create</button>
    </form>
  );
}
```

*Interview tip: Server Actions eliminate the need for separate API routes for mutations in Next.js.*

## Explain Next.js App Router vs Pages Router.

### Answer:

Pages Router (legacy): file-based routing in /pages, uses `getServerSideProps`/`getStaticProps`, no Server Components. App Router (modern): file-based in /app, Server Components by default, layouts, `loading.tsx`, `error.tsx`, streaming, and parallel routes. App Router is the future.

```
// Pages Router (old)
// pages/about.tsx
export async function getServerSideProps() {
  const data = await fetchData();
  return { props: { data } };
}
export default function About({ data }) { ... }

// App Router (new)
// app/about/page.tsx
export default async function About() {
  const data = await fetchData(); // direct await!
  return <div>{data.title}</div>;
}
// app/about/loading.tsx → auto Suspense
// app/about/error.tsx → auto Error Boundary
```

*Interview tip: New projects should always use App Router. Migrate existing Pages Router incrementally.*

## What are hydration errors and how do you fix them?

### Answer:

Hydration errors occur when server-rendered HTML doesn't match client-rendered output. React expects the initial client render to exactly match the server HTML. Common causes: using `Date.now()`, window checks, random values, or browser-only APIs during initial render.

```
// BAD – different on server vs client
function Clock() {
  return <p>{new Date().toLocaleString()}</p>;
}

// FIX – render only on client
function Clock() {
  const [time, setTime] = useState('');
  useEffect(() => {
    setTime(new Date().toLocaleString());
  }, []);
  return <p>{time}</p>;
}

// Or suppress hydration warning (last resort)
<p suppressHydrationWarning>
  {new Date().toLocaleString()}
</p>
```

*Interview tip: Server and client must produce identical initial HTML. Use `useEffect` for client-only values.*

## Q50



INTERMEDIATE

**Explain Next.js caching layers.****Answer:**

Next.js has 4 cache layers: 1) Request Memoization — dedup same fetch in one render. 2) Data Cache — persists fetch results across requests (revalidate to refresh). 3) Full Route Cache — caches rendered HTML for static pages. 4) Router Cache — caches visited routes in the browser for instant back/forward.

```
// Opt out of Data Cache (fresh every request)
fetch(url, { cache: 'no-store' });

// Revalidate every 60 seconds
fetch(url, { next: { revalidate: 60 } });

// On-demand revalidation
import { revalidatePath, revalidateTag } from 'next/cache';
await revalidatePath('/products');
await revalidateTag('products');

// Tag a fetch for targeted revalidation
fetch(url, { next: { tags: ['products'] } });
```

*Interview tip: Understand all 4 cache layers to debug 'why isn't my data updating?' — the #1 Next.js question.*

## Q51



INTERMEDIATE

**What is React 19's use() hook?****Answer:**

use() reads the value of a Promise or Context during render. Unlike useContext, it can be called conditionally. Unlike await, it integrates with Suspense — the component suspends while the promise resolves. It replaces many useEffect-for-data-fetching patterns.

```
import { use, Suspense } from 'react';

function UserProfile({ userPromise }) {
  const user = use(userPromise); // suspends!
  return <h1>{user.name}</h1>;
}

// Conditional context (not possible with useContext)
function Theme({ showTheme }) {
  if (showTheme) {
    const theme = use(ThemeContext);
    return <p>{theme}</p>;
  }
  return null;
}
```

*Interview tip: use() can be called inside if/else blocks — the only hook that breaks the 'no conditional hooks' rule.*

## Q52



INTERMEDIATE

**Explain React ref forwarding and useImperativeHandle.****Answer:**

forwardRef passes a ref from parent to a child's internal DOM element. useImperativeHandle customizes what the ref exposes — instead of the raw DOM node, you expose specific methods. This creates clean component APIs while hiding implementation details.

```
const FancyInput = forwardRef((props, ref) => {
  const inputRef = useRef();

  useImperativeHandle(ref, () => ({
    focus: () => inputRef.current.focus(),
    clear: () => { inputRef.current.value = ''; },
  }));

  return <input ref={inputRef} {...props} />;
});

// Parent
const ref = useRef();
<FancyInput ref={ref} />
ref.current.focus(); // calls exposed method
ref.current.clear(); // works too
```

*Interview tip: useImperativeHandle lets you expose only what's needed — better encapsulation than raw refs.*

**Q53**

★★★★★

**ADVANCED**

## Explain Next.js Middleware and its use cases.

### Answer:

Middleware runs on the Edge BEFORE any page renders — can redirect, rewrite URLs, set headers, and check auth. It executes at CDN edge nodes globally with near-zero latency. Perfect for auth guards, A/B testing, geolocation routing, and i18n redirects.

```
// middleware.ts
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(req: NextRequest) {
  const token = req.cookies.get('token');
  if (!token && req.nextUrl.pathname.startsWith('/dashboard')) {
    return NextResponse.redirect(
      new URL('/login', req.url)
    );
  }
  return NextResponse.next();
}

export const config = {
  matcher: ['/dashboard/:path*'],
};
```

*Interview tip: Middleware runs on the Edge Runtime — limited to Web APIs (no Node.js fs, child\_process, etc.).*

**Q54**

★★★★★

**ADVANCED**

## How do you implement code splitting in Next.js?

### Answer:

Next.js automatically code-splits by page. For component-level splitting, use next/dynamic (which wraps React.lazy + Suspense). Use ssr: false for browser-only components (charts, maps, editors). This reduces initial bundle size dramatically.

```
import dynamic from 'next/dynamic';

// Heavy chart only loaded when rendered
const Chart = dynamic(
  () => import('../components/Chart'),
  {

```

```

    loading: () => <p>Loading chart...</p>,
    ssr: false, // browser-only component
  }
);

export default function Dashboard() {
  return (
    <main>
      <h1>Dashboard</h1>
      <Chart data={data} />
    </main>
  );
}

```

*Interview tip: Use `ssr: false` for components using `window`, `document`, or other browser-only APIs.*

**Q55**

★★★★★

**ADVANCED**

## How would you implement authentication in Next.js?

### Answer:

Best practice: use Auth.js (NextAuth). For custom: store JWT in httpOnly cookies (not localStorage — XSS-safe). Use Middleware to protect routes globally. Server Components read cookies directly; Client Components use a session hook or Server Actions.

```

// Route handler – login
export async function POST(req: Request) {
  const { email, password } = await req.json();
  const user = await validateUser(email, password);
  if (!user)
    return Response.json({ error: 'Unauthorized' }, { status: 401 });

  const token = signJWT({ id: user.id, role: user.role });
  const res = Response.json({ ok: true });
  res.headers.set('Set-Cookie',
    `token=${token}; HttpOnly; Secure; SameSite=Strict; Path=/; Max-Age=3600`
  );
  return res;
}

```

*Interview tip: Never store JWTs in localStorage — they are vulnerable to XSS attacks. Always use httpOnly cookies.*

**Q56**

★★★★★

**ADVANCED**

## Explain Next.js generateMetadata for dynamic SEO.

### Answer:

generateMetadata is an async function that generates page-specific meta tags (title, description, Open Graph, Twitter cards) at the route level. It can fetch data to build dynamic metadata. This is essential for SEO in dynamic pages.

```

// app/products/[id]/page.tsx
export async function generateMetadata({ params }) {
  const product = await getProduct(params.id);
  return {
    title: product.name + ' | MyStore',
    description: product.description.slice(0, 160),
    openGraph: {
      title: product.name,
      images: [product.image],
    },
    twitter: {
      card: 'summary_large_image',

```

```

    },
  };
}

export default async function ProductPage({ params }) {
  const product = await getProduct(params.id);
  return <ProductView product={product} />;
}

```

*Interview tip: generateMetadata runs on the server and deduplicates data fetching with the page component.*

**Q57**

★★★★★

**ADVANCED**

## What are Web Vitals (LCP, CLS, INP) and why are they important?

### Answer:

Core Web Vitals are Google's user experience metrics that directly impact SEO rankings. LCP (Largest Contentful Paint) < 2.5s — loading speed. CLS (Cumulative Layout Shift) < 0.1 — visual stability. INP (Interaction to Next Paint) < 200ms — responsiveness.

```

import { useReportWebVitals } from 'next/web-vitals'

export function WebVitals() {
  useReportWebVitals((metric) => {
    if (metric.name === 'LCP') {
      console.log('LCP:', metric.value);
      // Send to analytics
    }
  })
}

```

*Interview tip: Next.js optimizes these automatically via next/image (CLS/LCP), next/font (CLS), and Server Components (INP).*

**Q58**

★★★★★

**ADVANCED**

## What are React 19's useOptimistic and useFormStatus?

### Answer:

useOptimistic shows optimistic UI updates immediately while the server processes. useFormStatus gives pending state of a parent form — useful for showing loading in submit buttons. Both eliminate common boilerplate for form interactions.

```

// useOptimistic
function Todos({ todos, addTodo }) {
  const [optimistic, addOptimistic] = useOptimistic(
    todos,
    (state, newTodo) => [...state, { ...newTodo, pending: true }]
  );
  async function handleAdd(formData) {
    addOptimistic({ text: formData.get('text') });
    await addTodo(formData); // server action
  }
}

// useFormStatus
function SubmitButton() {
  const { pending } = useFormStatus();
  return <button disabled={pending}>
    {pending ? 'Saving...' : 'Save'}
  </button>;
}

```

*Interview tip: useOptimistic makes apps feel instant — update UI first, then confirm with server.*

## Explain Next.js Parallel Routes and Intercepting Routes.

### Answer:

Parallel Routes (@slot convention) render multiple pages simultaneously in the same layout — like dashboards with independent panels. Intercepting Routes (..) convention) show a route in a modal while preserving the underlying page — like Instagram's photo modal on feed.

```
// Parallel Routes
// app/layout.tsx
export default function Layout({ children, analytics, team }) {
  return (
    <>
      {children}
      {analytics} { /* @analytics/page.tsx */ }
      {team}      { /* @team/page.tsx */ }
    </>
  );
}
// Each slot loads independently + has its own loading/error

// Intercepting Routes
// app/feed/(..)photo/[id]/page.tsx
// Shows photo in modal when navigating from /feed
// Direct URL /photo/[id] shows full page
```

*Interview tip: Parallel Routes are perfect for dashboards. Intercepting Routes for modal-based navigation.*

## Explain the React 19 Compiler (React Forget) and how it changes memoization.

### Answer:

The React 19 Compiler (formerly React Forget) is an auto-memoization compiler. It compiles React code to automatically insert the equivalent of useMemo and useCallback. This eliminates the need for developers to manually manage dependency arrays and optimizes render performance by default. It guarantees component and hook values are only re-computed when their inputs actually change.

```
// Before: Manual memoization
const MemoizedList = React.memo(({ items }) => {
  const sorted = useMemo(() => items.sort(), [items]);
  const onClick = useCallback(() => {}, []);
  return <div onClick={onClick}>{sorted.join(', ')}</div>;
});

// After: Just normal code, the compiler handles memoization!
const List = ({ items }) => {
  const sorted = items.sort();
  const onClick = () => {};
  return <div onClick={onClick}>{sorted.join(', ')}</div>;
}
```

*Interview tip: The compiler makes manual useMemo and useCallback obsolete in most cases, but you still need them for custom hook stability if not using the compiler yet.*

## Q61



ADVANCED

**Explain the React Server Component (RSC) payload and how serialization works.****Answer:**

The RSC payload is a compact, serialized representation of the React component tree returned from the server to the client. It contains the rendered virtual DOM structure, resolved props, and references to Client Component files (client references). Because this payload is sent over the network, all props passed from Server Components to Client Components must be serializable (no functions, classes, or symbols).

```
// Server Component
async function ServerPage() {
  const data = await db.fetch();
  // data is serialized to JSON-like structure in RSC payload
  return <ClientButton data={data} />;
}

// RSC payload includes:
// M1:client-reference-to-ClientButton.js
// J0:["div",null,["h1",null,"Title"],["$M1",{ "data":{"id":1}}]]
```

*Interview tip: If you pass a function or non-serializable object across the server-client boundary, React will throw a serialization error.*

## Q62



ADVANCED

**Explain the security features of React 19 Server Actions.****Answer:**

React 19 Server Actions are secure endpoints by default. They include: 1) Automatic CSRF protection (checks Origin headers on requests), 2) Closed-over variable encryption (variables in action closure are encrypted before sending to the client so they cannot be inspected or tampered with), 3) Safe action exports (Next.js automatically generates random, unpredictable action URLs so they cannot be guessed).

```
// app/actions.ts
'use server';

export async function deleteUser(id: string) {
  // 1. origin header verified to prevent CSRF
  // 2. 'id' is verified and validated
  // 3. User authorization must still be checked manually!
  const session = await getSession();
  if (session?.user.role !== 'admin') throw new Error('Unauthorized');
  await db.user.delete(id);
}
```

*Interview tip: Even with Server Actions security, you must still perform authentication and authorization checks inside the action body.*

## Q63



ADVANCED

**How do you implement a Multi-Zone architecture in Next.js?****Answer:**

Multi-Zone allows you to merge multiple independently deployed Next.js applications into a single domain (e.g. /blog on one app, /dashboard on another). You configure this in next.config.js using rewrites to forward requests from the main app (the gateway) to the independent zone apps. This is ideal for large organizations where separate teams own separate sections of the site.

```
// next.config.js (Main App / Gateway)
module.exports = {
  async rewrites() {
```

```

return [
  {
    source: '/blog',
    destination: 'https://blog-zone.internal/blog',
  },
  {
    source: '/blog/:path*',
    destination: 'https://blog-zone.internal/blog/:path*',
  },
];
},
};
};

```

*Interview tip: Ensure assets and CSS paths do not conflict by using unique basePath settings in each sub-app's config.*

**Q64**

★★★★★

**ADVANCED**

### Explain how Server-Sent Events (SSE) and Streaming work in Next.js Route Handlers.

#### Answer:

Streaming in Next.js leverages HTTP chunked transfer encoding, allowing the server to stream parts of the UI (via Suspense) or raw data (like AI responses) as they become available. For raw text/AI response streaming, you return a Response object initialized with a ReadableStream from a Route Handler. This is crucial for AI chat interfaces (e.g., ChatGPT style streaming).

```

// app/api/chat/route.ts
export async function POST(req: Request) {
  const encoder = new TextEncoder();
  const stream = new ReadableStream({
    async start(controller) {
      for (let i = 0; i < 5; i++) {
        controller.enqueue(encoder.encode(`Chunk ${i}\n`));
        await new Promise(r => setTimeout(r, 1000));
      }
      controller.close();
    }
  });
  return new Response(stream, {
    headers: { 'Content-Type': 'text/event-stream' }
  });
}

```

*Interview tip: Streaming reduces Time to First Byte (TTFB) and allows you to render static layouts while slow data streams in.*

## Phase 3: Node.js & Express

25 Q&As

Q65

★★★★★

BASIC

### How does Node.js handle async if it is single-threaded?

#### Answer:

Node.js delegates heavy work (file I/O, network, crypto) to libuv's thread pool and the OS kernel. The single-threaded event loop continuously checks for completed operations and fires callbacks. This is why Node handles thousands of concurrent connections efficiently — it never blocks waiting.

```
const fs = require('fs');

// Non-blocking – delegates to OS
fs.readFile('large.txt', 'utf8', (err, data) => {
  // Runs when I/O completes
  console.log('Done:', data.length);
});

console.log('Runs first – sync'); // runs immediately
```

*Interview tip: CPU-bound tasks block the event loop — use `worker_threads` or `child_process` for those.*

Q66

★★★★★

BASIC

### What is Express middleware and how does it work?

#### Answer:

Middleware are functions in a pipeline — each receives `req`, `res`, and `next()`. They execute in order, can modify request/response, and pass control via `next()`. Think of it as an assembly line: each station does its job and passes the product forward. Order matters!

```
const app = express();

// Logger middleware
app.use((req, res, next) => {
  console.log(req.method + ' ' + req.path);
  next();
});

// Auth middleware
app.use('/api', requireAuth);

// Error handler (4 params – must be last)
app.use((err, req, res, next) => {
  res.status(500).json({ error: err.message });
});
```

*Interview tip: Error-handling middleware MUST have exactly 4 parameters (`err`, `req`, `res`, `next`) to work.*

Q67

★★★★★

BASIC

### What is CORS and how do you configure it?

#### Answer:

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks requests from different origins by default. The server must explicitly whitelist allowed origins, methods, and headers. Without proper CORS, your frontend can't talk to your backend on a different domain.

```
const cors = require('cors');

// Allow specific origin (production)
app.use(cors({
  origin: 'https://myapp.com',
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  credentials: true,
  allowedHeaders: ['Content-Type', 'Authorization'],
}));

// Multiple origins
const allowedOrigins = ['https://myapp.com', 'https://admin.myapp.com'];
app.use(cors({
  origin: (origin, callback) => {
    if (!origin || allowedOrigins.includes(origin))
      callback(null, true);
    else callback(new Error('CORS blocked'));
  },
}));
```

*Interview tip: Never use cors() with no options in production — it allows ALL origins. Always whitelist.*

**Q68**

★★★★★

**INTERMEDIATE**

## How do you structure a large Node.js application?

### Answer:

Layered architecture: Routes (HTTP layer) → Controllers (req/res handling) → Services (business logic) → Repositories/Models (data access). Each layer only talks to the one below. This separation makes code testable, maintainable, and mockable.

```
// routes/user.routes.ts
router.post('/users',
  validate(createUserSchema),
  userController.create
);

// controllers/user.controller.ts
async create(req, res, next) {
  try {
    const user = await userService.create(req.body);
    res.status(201).json(user);
  } catch(e) { next(e); }
}

// services/user.service.ts
async create(data) {
  await this.verifyEmailUnique(data.email);
  return userRepository.save(data);
}
```

*Interview tip: Never put business logic in controllers or routes — it kills testability.*

**Q69**

★★★★★

**INTERMEDIATE**

## Explain JWT authentication flow.

### Answer:

5-step dance: 1) User submits credentials. 2) Server validates and signs a JWT with a secret. 3) Client stores and sends it in Authorization header. 4) Server verifies the token on protected routes. 5) Use short-lived access tokens (15min) + long-lived refresh tokens (7-30 days in httpOnly cookies).

```
// Sign on login
const token = jwt.sign(
  { userId: user.id, role: user.role },
  process.env.JWT_SECRET,
  { expiresIn: '15m' }
);

// Verify middleware
function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'No token' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch { res.status(401).json({ error: 'Invalid token' }); }
}
```

*Interview tip: Access token: 15 min in memory/header. Refresh token: 7-30 days in httpOnly cookie.*

**Q70**

★★★★★

**INTERMEDIATE**

## How do you handle errors globally in Express?

### Answer:

Three-part strategy: 1) Custom error classes with status codes. 2) Async wrapper (catchAsync) to forward promise rejections. 3) Centralized error handler as the LAST middleware. Never send stack traces to clients in production.

```
// Custom error class
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
  }
}

// Async wrapper – catches promise rejections
const catchAsync = fn =>
  (req, res, next) =>
    Promise.resolve(fn(req, res, next)).catch(next);

// Global handler
app.use((err, req, res, next) => {
  const status = err.statusCode || 500;
  res.status(status).json({
    error: err.message,
    ...(process.env.NODE_ENV !== 'production'
      && { stack: err.stack })
  });
});
```

*Interview tip: Express only catches sync errors automatically — always handle async with catchAsync or try/catch.*

**Q71**

★★★★★

**INTERMEDIATE**

## What is RBAC and how do you implement it in Express?

### Answer:

Role-Based Access Control restricts API access based on user roles. Each role has a set of permissions. Middleware checks the user's role before granting access. This decouples authorization logic from business logic and makes it composable.

```
const permissions = {
  admin: ['read', 'write', 'delete'],
  editor: ['read', 'write'],
  viewer: ['read'],
};

function authorize(...perms) {
  return (req, res, next) => {
    const userPerms = permissions[req.user.role] || [];
    const allowed = perms.every(p => userPerms.includes(p));
    if (!allowed)
      return res.status(403).json({ error: 'Forbidden' });
    next();
  };
}

router.delete('/posts/:id',
  authenticate,
  authorize('delete'),
  postController.delete
);
```

*Interview tip: For complex permission systems, consider Casbin or OPA (Open Policy Agent).*

**Q72**

★★★★★

**INTERMEDIATE**

### Explain process.nextTick() vs setImmediate().

#### Answer:

process.nextTick runs BEFORE the next event loop phase (highest priority microtask). setImmediate runs AFTER the current poll phase completes (next iteration). nextTick can starve I/O if overused because it always cuts the line. In practice: use setImmediate for deferring, nextTick for critical callbacks.

```
// nextTick runs first
setImmediate(() => console.log('setImmediate'));
process.nextTick(() => console.log('nextTick'));
// Output: nextTick, setImmediate

// Danger: recursive nextTick starves I/O
function badIdea() {
  process.nextTick(badIdea); // blocks everything!
}

// Safe: recursive setImmediate allows I/O
function safe() {
  setImmediate(safe); // I/O can interleave
}
```

*Interview tip: Use setImmediate to yield to I/O. Use nextTick only when you need to run before any I/O.*

**Q73**

★★★★★

**INTERMEDIATE**

### How do you handle file uploads in Node.js?

#### Answer:

Use Multer middleware for multipart/form-data (file uploads). Configure storage (disk or memory), file size limits, and file type filters. For large files, stream directly to cloud storage (S3) instead of loading into memory.

```
const multer = require('multer');

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, 'uploads/'),
```

```

    filename: (req, file, cb) => {
      const unique = Date.now() + '-' + Math.round(Math.random() * 1E9);
      cb(null, unique + '-' + file.originalname);
    }
  });

const upload = multer({
  storage,
  limits: { fileSize: 5 * 1024 * 1024 }, // 5MB
  fileFilter: (req, file, cb) => {
    if (file.mimetype.startsWith('image/')) cb(null, true);
    else cb(new Error('Only images allowed'), false);
  }
});

app.post('/avatar', upload.single('photo'), (req, res) => {
  res.json({ url: '/uploads/' + req.file.filename });
});

```

*Interview tip: For production, stream uploads directly to S3/GCS — never store files on the app server.*

**Q74**

★★★★★

**INTERMEDIATE**

**Explain the Node.js cluster module and worker threads.**

**Answer:**

Cluster: forks multiple processes (one per CPU core) sharing the same port — each process has its own event loop and memory. Worker threads: share memory via SharedArrayBuffer — true multi-threading for CPU-intensive work. Use cluster for scaling HTTP servers, workers for CPU-bound tasks.

```

// Cluster – scale HTTP server
const cluster = require('cluster');
const os = require('os');

if (cluster.isPrimary) {
  const cpus = os.cpus().length;
  for (let i = 0; i < cpus; i++) cluster.fork();
  cluster.on('exit', () => cluster.fork());
} else {
  app.listen(3000);
}

// Worker thread – CPU-intensive task
const { Worker } = require('worker_threads');
const worker = new Worker('./heavy-task.js');
worker.on('message', result => console.log(result));

```

*Interview tip: In production, use PM2 in cluster mode instead of manual cluster management.*

**Q75**

★★★★★

**INTERMEDIATE**

**What are security headers and how do you set them?**

**Answer:**

Security headers tell browsers how to protect your app. Helmet.js sets them automatically: Content-Security-Policy (blocks XSS), X-Frame-Options (prevents clickjacking), Strict-Transport-Security (forces HTTPS), X-Content-Type-Options (prevents MIME sniffing).

```

const helmet = require('helmet');
app.use(helmet()); // sets all security headers

// Custom CSP

```

```
app.use(helmet.contentSecurityPolicy({
  directives: {
    defaultSrc: ['"self"'],
    scriptSrc: ['"self"', "'unsafe-inline'"],
    styleSrc: ['"self"', "fonts.googleapis.com"],
    imgSrc: ['"self"', "data:", "cdn.example.com"],
    connectSrc: ['"self"', "api.example.com"],
  }
}));
```

*Interview tip: Always use Helmet in production. One line (app.use(helmet())) gives you major security improvements.*

**Q76**

★★★★★

**INTERMEDIATE**

## How do you implement graceful shutdown in Node.js?

### Answer:

Graceful shutdown lets the server finish active requests before stopping — no dropped connections. Listen for SIGTERM/SIGINT, stop accepting new connections, wait for in-flight requests to complete, then close database pools and exit. Critical for zero-downtime deployments.

```
process.on('SIGTERM', gracefulShutdown);
process.on('SIGINT', gracefulShutdown);

async function gracefulShutdown(signal) {
  console.log(`${signal} received. Shutting down...`);

  // Stop accepting new connections
  server.close(async () => {
    console.log('HTTP server closed');

    // Close DB connections
    await db.end();
    await redis.quit();

    console.log('All connections closed');
    process.exit(0);
  });

  // Force kill after 30 seconds
  setTimeout(() => {
    console.error('Forced shutdown');
    process.exit(1);
  }, 30000);
}
```

*Interview tip: Kubernetes sends SIGTERM before killing pods — graceful shutdown prevents request failures.*

**Q77**

★★★★★

**INTERMEDIATE**

## Explain Node.js EventEmitter pattern.

### Answer:

EventEmitter is Node's core pub/sub system — objects emit named events, listeners subscribe to them. It decouples producers from consumers. Most Node.js core modules (streams, http, fs) extend EventEmitter. It's the foundation of Node's event-driven architecture.

```
const EventEmitter = require('events');

class OrderService extends EventEmitter {
  async createOrder(data) {
    const order = await db.orders.create(data);
```

```

    this.emit('order:created', order);
    return order;
  }
}

const orders = new OrderService();

// Decoupled listeners
orders.on('order:created', order => {
  emailService.sendConfirmation(order);
});
orders.on('order:created', order => {
  analyticsService.track('purchase', order);
});

```

*Interview tip: EventEmitter is synchronous by default — listeners run in order, not in parallel.*

**Q78**

★★★★★

**INTERMEDIATE**

## Compare Express vs Fastify vs Hono.

### Answer:

Express: most popular, huge ecosystem, callback-based, moderate performance. Fastify: 2-3x faster than Express, schema-based validation, built-in serialization, plugin architecture. Hono: ultralight, runs everywhere (Node, Deno, Bun, Edge, Workers), Web Standard APIs. Choose based on platform and performance needs.

```

// Express
app.get('/hello', (req, res) => {
  res.json({ hello: 'world' });
});

// Fastify
fastify.get('/hello', {
  schema: {
    response: { 200: { type: 'object',
      properties: { hello: { type: 'string' }}} }
  }, async () => ({ hello: 'world' }));

// Hono (edge-native)
const app = new Hono();
app.get('/hello', (c) => c.json({ hello: 'world' }));

```

*Interview tip: Express for most projects. Fastify for high-performance APIs. Hono for edge/serverless.*

**Q79**

★★★★★

**ADVANCED**

## How do you implement rate limiting in a distributed Node.js system?

### Answer:

express-rate-limit handles single-server limits. For distributed systems (multiple servers behind a load balancer), use Redis as a shared store so rate limits apply consistently across all instances. Apply stricter limits on auth routes to prevent brute-force attacks.

```

const rateLimit = require('express-rate-limit');
const RedisStore = require('rate-limit-redis');

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100,
  standardHeaders: true,
  store: new RedisStore({

```

```

    client: redisClient,
    prefix: 'rl:',
  }},
  keyGenerator: req => req.user?.id || req.ip,
});

// Stricter limit on auth routes
const authLimiter = rateLimit({
  windowMs: 60 * 1000,
  max: 5, // 5 attempts per minute
});

app.use('/api/', limiter);
app.use('/api/auth/', authLimiter);

```

*Interview tip: Apply stricter limits on /login and /register to prevent brute-force attacks.*

**Q80**

★★★★★

**ADVANCED**

## Explain Node.js streams and when to use them.

### Answer:

Streams process data in chunks rather than loading everything into memory. 4 types: Readable, Writable, Duplex (both), Transform (modify data as it passes). Critical for large files, video streaming, and data pipelines. A 2GB file doesn't need 2GB RAM — streams use constant memory.

```

const fs = require('fs');
const zlib = require('zlib');

// Stream large file through gzip compression
fs.createReadStream('large.csv')
  .pipe(zlib.createGzip())
  .pipe(fs.createWriteStream('large.csv.gz'))
  .on('finish', () => console.log('Compressed'));

// Stream DB query results to HTTP response
app.get('/export', (req, res) => {
  res.setHeader('Content-Type', 'text/csv');
  db.queryStream('SELECT * FROM orders')
    .pipe(csvTransform)
    .pipe(res);
});

```

*Interview tip: Never use fs.readFile for large files — always stream them.*

**Q81**

★★★★★

**ADVANCED**

## How do you implement WebSockets for real-time features?

### Answer:

WebSockets provide full-duplex persistent connections — the server can push data to clients instantly. Unlike HTTP polling (client asks repeatedly), WebSockets stay open. Use Socket.IO for production (handles reconnection, rooms, fallbacks). Scale across servers with Redis pub/sub adapter.

```

const { Server } = require('socket.io');
const io = new Server(httpServer);

io.on('connection', socket => {
  socket.on('join-room', roomId => {
    socket.join(roomId);
    socket.to(roomId).emit('user-joined', socket.id);
  });
});

```

```

socket.on('message', ({ roomId, text }) => {
  io.to(roomId).emit('message', {
    from: socket.id,
    text,
    at: new Date().toISOString()
  });
});

socket.on('disconnect', () => {
  console.log('Disconnected:', socket.id);
});
});

```

*Interview tip: Use `@socket.io/redis-adapter` to sync events across multiple Node.js instances.*

**Q82**

★★★★★

**ADVANCED**

## How do you manage environment variables securely?

### Answer:

Never hardcode secrets. Use `.env` files locally (dotenv), but in production use secret managers (AWS Secrets Manager, Vault). Validate all env vars at startup — fail fast if any are missing. Never commit `.env` to git.

```

// Validate env vars at startup
const required = ['DATABASE_URL', 'JWT_SECRET', 'REDIS_URL'];
for (const key of required) {
  if (!process.env[key])
    throw new Error(`Missing required env var: ${key}`);
}

// Use a config module
export const config = {
  port: parseInt(process.env.PORT || '3000'),
  db: process.env.DATABASE_URL,
  jwt: {
    secret: process.env.JWT_SECRET,
    expiresIn: process.env.JWT_EXPIRES || '15m',
  },
  isProduction: process.env.NODE_ENV === 'production',
} as const;

```

*Interview tip: Use zod to validate env vars: `z.object({ PORT: z.coerce.number() }).parse(process.env)`.*

**Q83**

★★★★★

**ADVANCED**

## What is input validation and how do you implement it?

### Answer:

Never trust client input — validate everything on the server. Use schema validation libraries (Zod, Joi, Yup) to define expected shapes and reject invalid data early. Validate in middleware before the request reaches controllers.

```

const { z } = require('zod');

const createUserSchema = z.object({
  name: z.string().min(2).max(100),
  email: z.string().email(),
  age: z.number().int().min(18).optional(),
  role: z.enum(['user', 'admin']).default('user'),
});

// Validation middleware

```

```
function validate(schema) {
  return (req, res, next) => {
    const result = schema.safeParse(req.body);
    if (!result.success) {
      return res.status(400).json({
        errors: result.error.flatten().fieldErrors
      });
    }
    req.body = result.data; // cleaned data
    next();
  };
}

app.post('/users', validate(createUserSchema), controller.create);
```

*Interview tip: Zod is the standard for TypeScript projects — it infers TypeScript types from schemas.*

**Q84**

★★★★★

**ADVANCED**

**Explain the phases of the Node.js Event Loop in detail.**

**Answer:**

The event loop has six main phases: 1) Timers (executes setTimeout/setInterval callbacks), 2) Pending callbacks (executes I/O callbacks deferred from previous loop iteration), 3) Idle, prepare (internal use), 4) Poll (retrieves new I/O events; executes I/O-related callbacks), 5) Check (executes setImmediate callbacks), 6) Close callbacks (e.g., socket.on('close')). Between each phase, Node.js processes microtasks (Promises and process.nextTick).

```
setTimeout(() => console.log('Timer'), 0);
setImmediate(() => console.log('Immediate'));
process.nextTick(() => console.log('NextTick'));
Promise.resolve().then(() => console.log('Promise'));

// Output order:
// 1. NextTick
// 2. Promise
// 3. Timer
// 4. Immediate (depending on loop entrance timing)
```

*Interview tip: process.nextTick fires immediately after the current operation finishes, before event loop phases run.*

**Q85**

★★★★★

**ADVANCED**

**How do you detect and debug memory leaks in Node.js applications?**

**Answer:**

Memory leaks happen when objects are kept in memory and not garbage collected. To debug: 1) Run Node with the --inspect flag, 2) Use Chrome DevTools or a tool like memlab to capture Heap Snapshots, 3) Perform actions to trigger the leak, 4) Take another snapshot and compare them (using 'Comparison' view to find growing object counts). Common culprits include global variables, forgotten event listeners, intervals, and closures.

```
// Start node with inspect: node --inspect index.js
// Programmatically triggering a heap snapshot:
const heapdump = require('heapdump');

heapdump.writeSnapshot((err, filename) => {
  console.log('Snapshot written to:', filename);
});
```

*Interview tip: Check for growing array/object lengths stored outside request lifecycle context.*

## Q86



ADVANCED

### What is AsyncLocalStorage and how is it used for request tracking?

**Answer:**

AsyncLocalStorage (from the `node:async_hooks` module) lets you store data throughout the lifetime of an asynchronous web request, similar to thread-local storage in multi-threaded languages. This is incredibly useful for passing request-scoped context (like a user ID or Correlation ID for logging) to deeply nested functions without explicitly passing it through every function argument (prop drilling).

```
const { AsyncLocalStorage } = require('node:async_hooks');
const storage = new AsyncLocalStorage();

app.use((req, res, next) => {
  const requestId = req.headers['x-request-id'] || generateId();
  storage.run({ requestId }, () => next());
});

// Deep in controller or DB layer:
function logMessage(msg) {
  const store = storage.getStore();
  console.log(`[${store?.requestId || 'system'}] ${msg}`);
}
```

*Interview tip: AsyncLocalStorage is standard in modern frameworks (like Next.js headers/cookies functions).*

## Q87



ADVANCED

### How do you implement structured logging and distributed tracing in Node.js?

**Answer:**

Structured logging uses JSON format instead of plain text, making logs easily searchable by log aggregators (Elasticsearch, Loki, Datadog). Use a fast logger like Pino or Winston. Combined with AsyncLocalStorage, you can inject a Correlation ID into every log statement inside the request lifecycle to trace requests across microservices.

```
const pino = require('pino');
const logger = pino();

// Structured log output:
// {'level':30,'time':1623457,'msg':'User logged in','userId':123}
logger.info({ userId: 123 }, 'User logged in');

// Downstream requests propagate correlation ID:
// fetch('http://payment-service/pay', {
//   headers: { 'X-Correlation-ID': correlationId }
// });
```

*Interview tip: Never log sensitive data (passwords, credit cards) — use logger level sanitizers to filter them.*

## Q88



INTERMEDIATE

### Explain package manager differences: npm vs yarn vs pnpm.

**Answer:**

npm/yarn v1 use flat `node_modules`, which leads to 'phantom dependencies' (packages using dependencies not declared in `package.json` due to hoisting) and slow, duplicate storage. pnpm solves this using a content-addressable store (files are stored once globally in `~/.pnpm-store` and hard-linked into projects) and a nested `node_modules` structure using symlinks, which enforces dependency declaration strictly and saves disk space.

```
# npm/Yarn v1: flat folder structure
# node_modules/express, node_modules/accepts (hoisted!)

# pnpm: symlinked nested folder structure
# node_modules/express -> symlink to .pnpm/express@x.y.z/node_modules/express
# Prevents code from importing 'accepts' unless explicitly declared.
```

*Interview tip: Use pnpm for faster installs, disk space savings, and to avoid phantom dependency bugs.*

**Q89**

★★★★★

**ADVANCED**

## How do you secure Node.js applications against OWASP Top 10 vulnerabilities?

### Answer:

1) Prototype Pollution: freeze object prototypes or use `Object.create(null)` for untrusted keys. 2) SQL/NoSQL Injection: use parameterized queries (ORM/ODM) and sanitize inputs. 3) XSS: escape HTML output and use Helmet middleware for secure HTTP headers. 4) Dependency vulnerabilities: regularly run `npm audit` or use `Snyk/Dependabot`.

```
// 1. helmet for security headers
const helmet = require('helmet');
app.use(helmet());

// 2. Safe objects (prevents prototype pollution)
const obj = Object.create(null);

// 3. Parameterized query (prevents SQLi)
db.query('SELECT * FROM users WHERE id = $1', [userId]);
```

*Interview tip: Regularly audit production dependencies using `npm audit` or `yarn audit` to patch CVEs.*

## Phase 4: MongoDB & PostgreSQL

22 Q&As

Q90

★★★★★

BASIC

### When should you use MongoDB vs PostgreSQL?

#### Answer:

MongoDB: flexible documents, nested/hierarchical data, rapidly changing schemas, horizontal scaling. PostgreSQL: relational data, complex joins, ACID transactions, strong consistency. Rule: if data has relationships and needs integrity → PostgreSQL. If schema is fluid and data is nested → MongoDB.

```
// MongoDB – flexible nested document
{
  _id: '123',
  name: 'Nazmul',
  addresses: [
    { type: 'home', city: 'Dhaka' },
    { type: 'work', city: 'Mirpur' }
  ]
}
```

```
// PostgreSQL – normalized
-- users: id, name
-- addresses: id, user_id FK, type, city
```

*Interview tip: Need ACID transactions and complex joins? PostgreSQL. Need schema flexibility and scale? MongoDB.*

Q91

★★★★★

BASIC

### What are database indexes and why do they matter?

#### Answer:

Indexes are like a book's index — instead of reading every page (full table scan), you jump directly to the right page. Without an index:  $O(n)$  scan. With an index (B-tree):  $O(\log n)$ . Trade-off: indexes speed reads but slow writes and use storage. Index columns you query often.

```
-- PostgreSQL
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_posts_user_created
  ON posts(user_id, created_at DESC); -- composite

-- MongoDB
db.users.createIndex({ email: 1 }, { unique: true });
db.orders.createIndex(
  { userId: 1, createdAt: -1 }
);
```

*Interview tip: Use EXPLAIN ANALYZE (Postgres) or .explain('executionStats') (MongoDB) to verify index usage.*

Q92

★★★★★

BASIC

### What is database normalization?

#### Answer:

Normalization organizes data to reduce redundancy and improve integrity. 1NF: no repeating groups (each cell has one value). 2NF: 1NF + no partial dependencies. 3NF: 2NF + no transitive dependencies. In practice, normalize first, then selectively denormalize for read performance.

```
-- UNNORMALIZED (bad)
-- orders: id, customer_name, customer_email, product, price

-- NORMALIZED (3NF)
-- customers: id, name, email
-- products: id, name, price
-- orders: id, customer_id FK, product_id FK, quantity

-- Denormalization (for read perf)
-- order_summaries: id, customer_name, total
-- Materialized view or cache
```

*Interview tip: Normalize for writes (data integrity), denormalize for reads (query speed). Balance both.*

**Q93**

★★★★★

**INTERMEDIATE**

## What is the N+1 query problem and how do you solve it?

### Answer:

N+1 occurs when you fetch N records then make 1 extra query per record to get related data. 100 posts + 1 author query each = 101 queries. Fix: SQL JOINS, ORM eager loading (include/populate), or DataLoader batching in GraphQL.

```
// N+1 problem
const posts = await Post.findAll(); // 1 query
for (const post of posts) {
  // N queries!
  const author = await User.findById(post.userId);
}

// Fix: eager load (1 JOIN query)
const posts = await Post.findAll({
  include: [{ model: User, as: 'author' }]
});

// MongoDB fix: $lookup
db.posts.aggregate([
  { $lookup: {
    from: 'users', localField: 'userId',
    foreignField: '_id', as: 'author'
  } }
]);
```

*Interview tip: Enable query logging in development to detect N+1 — Mongoose debug mode, Sequelize logging: true.*

**Q94**

★★★★★

**INTERMEDIATE**

## Explain MongoDB's aggregation pipeline.

### Answer:

The aggregation pipeline processes documents through sequential stages, each transforming the output for the next. Think of it as a Unix pipe: data flows through stages. Key stages: \$match (filter), \$group (aggregate), \$project (reshape), \$sort, \$lookup (join), \$unwind (flatten arrays).

```
db.orders.aggregate([
  { $match: { status: 'completed' } },
  { $group: {
    _id: '$userId',
    totalSpent: { $sum: '$amount' },
    orderCount: { $sum: 1 }
  } },
  { $lookup: {
    from: 'users',
```

```

    localField: '_id',
    foreignField: '_id',
    as: 'user'
  }},
  { $unwind: '$user' },
  { $sort: { totalSpent: -1 } },
  { $limit: 10 }
]);

```

*Interview tip: Always put \$match as early as possible in the pipeline to reduce document count in later stages.*

**Q95**

★★★★★

**INTERMEDIATE**

## What are PostgreSQL transactions and ACID properties?

### Answer:

ACID — the 4 guarantees of reliable transactions: Atomicity (all or nothing), Consistency (valid state before/after), Isolation (concurrent transactions don't interfere), Durability (committed data survives crashes). If any operation fails, everything rolls back.

```

BEGIN;

UPDATE accounts
  SET balance = balance - 500
  WHERE id = 1;

UPDATE accounts
  SET balance = balance + 500
  WHERE id = 2;

COMMIT; -- Both succeed or both roll back

-- Savepoints for partial rollback
SAVEPOINT before_step;
-- ... more operations ...
ROLLBACK TO SAVEPOINT before_step;

```

*Interview tip: MongoDB 4.0+ supports multi-document ACID transactions, but they add overhead vs SQL.*

**Q96**

★★★★★

**INTERMEDIATE**

## Compare Prisma vs Drizzle ORM.

### Answer:

Prisma: declarative schema (schema.prisma), auto-generated type-safe client, great DX, migrations built-in, slightly heavier. Drizzle: SQL-like syntax in TypeScript, lighter weight, closer to raw SQL, better for edge/serverless. Prisma for productivity, Drizzle for control and performance.

```

// Prisma – declarative
const user = await prisma.user.findUnique({
  where: { email: 'test@test.com' },
  include: { posts: true },
});

// Drizzle – SQL-like
const user = await db
  .select()
  .from(users)
  .where(eq(users.email, 'test@test.com'))
  .leftJoin(posts, eq(users.id, posts.userId));

```

*Interview tip: Prisma for rapid development. Drizzle for serverless/edge where cold start matters.*

## Explain PostgreSQL JSON/JSONB columns.

### Answer:

PostgreSQL can store JSON data in columns — JSONB is the binary format (faster queries, indexable). This gives you document-like flexibility within a relational database. You can query, index, and validate JSON fields. Best of both worlds: relational + flexible.

```
-- Create with JSONB
CREATE TABLE products (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  attributes JSONB DEFAULT '{}')
);

-- Insert
INSERT INTO products (name, attributes)
VALUES ('Laptop', '{"ram": 16, "color": "silver"}');

-- Query JSONB
SELECT * FROM products
WHERE attributes->>'color' = 'silver';

-- GIN index for JSONB
CREATE INDEX idx_prod_attrs ON products
  USING GIN (attributes);
```

*Interview tip: Use JSONB (not JSON) — it's parsed on write, faster on read, and supports indexing.*

## How do you handle database migrations?

### Answer:

Migrations are version-controlled schema changes — each migration file describes a forward change (up) and a rollback (down). They ensure every environment has the same schema. Never modify production schema manually — always use migrations.

```
// Prisma migration
// prisma/migrations/001_add_posts/migration.sql
CREATE TABLE posts (
  id SERIAL PRIMARY KEY,
  title TEXT NOT NULL,
  content TEXT,
  author_id INT REFERENCES users(id),
  created_at TIMESTAMP DEFAULT NOW())
);

// CLI commands
// npx prisma migrate dev --name add_posts
// npx prisma migrate deploy (production)
// npx prisma migrate reset (dev only)

// Knex.js migration
exports.up = (knex) => knex.schema.createTable('posts', t => {
  t.increments('id');
  t.string('title').nullable();
});
```

*Interview tip: Always test migrations on a staging database before running in production.*

## Explain MongoDB schema design: embedding vs referencing.

### Answer:

Embed when: data is always accessed together, bounded one-to-few, rarely changes independently.  
Reference when: many-to-many, accessed independently, unbounded growth, or documents risk exceeding 16MB. Remember: embed for reads, reference for writes.

```
// Embed – post with few comments
{
  _id: 'post1',
  title: 'Hello World',
  comments: [
    { author: 'Ali', text: 'Great!' }
  ]
}

// Reference – user with millions of orders
// users: { _id, name, email }
// orders: { _id, userId: ObjectId, amount }

// Query with $lookup
db.orders.aggregate([
  { $lookup: {
    from: 'users',
    localField: 'userId',
    foreignField: '_id',
    as: 'user'
  } }
]);
```

*Interview tip: MongoDB documents have a hard 16MB limit — never embed unbounded arrays.*

## What is full-text search and how do databases support it?

### Answer:

Full-text search finds documents by words/phrases with relevance ranking — unlike LIKE which does pattern matching. PostgreSQL has built-in tsvector/tsquery. MongoDB has text indexes. For advanced needs (fuzzy, faceted, autocomplete), use Elasticsearch.

```
-- PostgreSQL full-text search
ALTER TABLE articles ADD COLUMN search_vector tsvector;
UPDATE articles SET search_vector =
  to_tsvector('english', title || ' ' || body);
CREATE INDEX idx_search ON articles USING GIN(search_vector);

SELECT title, ts_rank(search_vector, query) AS rank
FROM articles, to_tsquery('react & hooks') query
WHERE search_vector @@ query
ORDER BY rank DESC;

// MongoDB text search
db.articles.createIndex({ title: 'text', body: 'text' });
db.articles.find({ $text: { $search: 'react hooks' } });
```

*Interview tip: PostgreSQL full-text search is surprisingly powerful — try it before adding Elasticsearch.*

### Explain MongoDB change streams.

#### Answer:

Change streams let you watch a collection for real-time changes (insert, update, delete) — like database triggers but application-level. Built on the oplog. Perfect for real-time sync, cache invalidation, and event-driven architectures.

```
const pipeline = [
  { $match: { operationType: { $in: ['insert', 'update'] } } },
  { $match: { 'fullDocument.status': 'published' } }
];

const changeStream = db.collection('posts').watch(pipeline);

changeStream.on('change', (change) => {
  switch (change.operationType) {
    case 'insert':
      notifySubscribers(change.fullDocument);
      break;
    case 'update':
      invalidateCache(change.documentKey._id);
      break;
  }
});
```

*Interview tip: Change streams require a replica set — they won't work on standalone MongoDB instances.*

### Explain read replicas and database sharding.

#### Answer:

Read Replicas: copies of the primary DB that handle read traffic. Writes go to primary, reads go to replicas — reduces read pressure. Sharding: partitions data across multiple databases by shard key. Each shard holds a subset. Together they enable massive horizontal scaling.

```
// Read/write separation
await primaryPool.query('INSERT INTO orders...');
await replicaPool.query('SELECT * FROM orders WHERE userId = $1', [id]);

// MongoDB sharding
sh.enableSharding('mydb');
sh.shardCollection('mydb.users', { region: 'hashed' });
// Asia users -> Shard A
// Europe users -> Shard B
// Queries route automatically
```

*Interview tip: Choose shard keys carefully — a bad key causes hot spots where one shard gets all the traffic.*

### How do you optimize a slow SQL query?

#### Answer:

7-step optimization: 1) EXPLAIN ANALYZE — see the query plan. 2) Check for missing indexes. 3) Avoid SELECT \* — only fetch needed columns. 4) Don't wrap indexed columns in functions. 5) Paginate large results. 6) Use covering indexes. 7) Cache hot queries in Redis.

```
-- Slow: function prevents index use
SELECT * FROM users
WHERE LOWER(email) = 'test@example.com';
```

```
-- Fix: index on expression
CREATE INDEX idx_lower_email
  ON users(LOWER(email));

-- Better projection
SELECT id, name, email FROM users
WHERE LOWER(email) = 'test@example.com';

-- Check plan
EXPLAIN ANALYZE
SELECT id, name FROM users
WHERE email = 'x@y.com';
```

*Interview tip: 'Seq Scan' in EXPLAIN output = no index used. 'Index Scan' = good. 'Bitmap Heap Scan' = ok.*

**Q104**

★★★★★

**ADVANCED**

### Explain database connection pooling.

#### Answer:

Creating a new DB connection is expensive (TCP handshake, auth, SSL, memory allocation). Connection pooling maintains a pool of pre-established connections reused across requests. Without pooling, high traffic exhausts DB connections and crashes the database.

```
// PostgreSQL with pg pool
const { Pool } = require('pg');

const pool = new Pool({
  host: process.env.DB_HOST,
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  max: 20,           // max connections
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
});

// Reuses existing connection from pool
const result = await pool.query(
  'SELECT * FROM users WHERE id = $1', [id]
);
```

*Interview tip: Tune max pool size based on DB max\_connections. Rule: pool size = (number of cores \* 2) + effective spindle count.*

**Q105**

★★★★★

**ADVANCED**

### What are PostgreSQL Common Table Expressions (CTEs)?

#### Answer:

CTEs (WITH clauses) are named temporary result sets within a query. They make complex queries readable by breaking them into logical steps. Recursive CTEs handle tree/hierarchical data (org charts, category trees). Think of CTEs as 'query variables'.

```
-- Regular CTE
WITH active_users AS (
  SELECT id, name FROM users WHERE active = true
),
user_orders AS (
  SELECT user_id, SUM(amount) as total
  FROM orders GROUP BY user_id
)
```

```

SELECT u.name, COALESCE(o.total, 0) as total
FROM active_users u
LEFT JOIN user_orders o ON u.id = o.user_id;

-- Recursive CTE (org chart)
WITH RECURSIVE org AS (
  SELECT id, name, manager_id, 0 as depth
  FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, o.depth + 1
  FROM employees e JOIN org o ON e.manager_id = o.id
)
SELECT * FROM org;

```

*Interview tip: Recursive CTEs replace what would be loops in application code — let the database handle it.*

**Q106**

★★★★★

ADVANCED

## How do you handle database backup and disaster recovery?

### Answer:

Three strategies: 1) Logical backups (pg\_dump/mongodump) — portable but slow for large DBs. 2) Physical backups (file system snapshots, pg\_basebackup) — fast but DB-specific. 3) Point-in-time recovery (WAL archiving / oplog) — restore to any moment. Test restores regularly!

```

# PostgreSQL backup
pg_dump -Fc mydb > backup.dump
pg_restore -d mydb backup.dump

# Automated with cron
0 2 * * * pg_dump -Fc mydb > /backups/mydb_$(date +%Y%m%d).dump

# MongoDB backup
mongodump --uri='mongodb://...' --out=/backups/$(date +%Y%m%d)
mongorestore --uri='mongodb://...' /backups/20250101

# Point-in-time recovery (PostgreSQL WAL)
archive_mode = on
archive_command = 'cp %p /wal_archive/%f'

```

*Interview tip: An untested backup is not a backup. Regularly practice restoring from backups.*

**Q107**

★★★★★

ADVANCED

## Explain database locking mechanisms (Shared, Exclusive, Optimistic, Pessimistic).

### Answer:

Locks prevent race conditions. Shared lock (S) allows multiple reads but blocks writes. Exclusive lock (X) blocks both reads and writes. Pessimistic locking locks rows when reading (SELECT ... FOR UPDATE) until transaction completes. Optimistic locking uses a version column and checks if it has changed at update time, retrying on conflict.

```

-- Pessimistic lock (blocks until current transaction completes)
BEGIN;
SELECT * FROM inventory WHERE item_id = 1 FOR UPDATE;
UPDATE inventory SET stock = stock - 1 WHERE item_id = 1;
COMMIT;

-- Optimistic lock (no database-level locking)
UPDATE inventory SET stock = stock - 1, version = version + 1
WHERE item_id = 1 AND version = 5; -- fails if version changed!

```

*Interview tip: Pessimistic locking is better for high-contention, write-heavy rows. Optimistic locking is better for read-heavy rows.*

**Q108**

★★★★★

**INTERMEDIATE**

**Explain the CAP Theorem and how it applies to MongoDB and PostgreSQL.**

**Answer:**

The CAP Theorem states a distributed system can guarantee at most two of: Consistency, Availability, Partition tolerance. PostgreSQL is traditionally a CA system (prioritizes ACID consistency/availability, though partitions cause issues). MongoDB is a CP system (Consistency + Partition tolerance) by default: if a network partition isolates the primary node, secondaries refuse writes while electing a new primary.

```
// MongoDB CP setting: replica sets elect new primary.
// During elections, writes are blocked (losing Availability).
// Read preference can be tuned to read stale data (prioritizing Availability):
const cursor = db.collection('posts').find({}).readPref('secondary');
```

*Interview tip: No system is 100% Consistent and 100% Available during a network partition (P) — you must choose C or A.*

**Q109**

★★★★★

**ADVANCED**

**Explain PostgreSQL Window Functions with examples.**

**Answer:**

Window functions perform calculations across a set of table rows related to the current row without grouping them into a single output row. They keep individual row identities intact. Syntax uses the OVER clause, often combined with PARTITION BY and ORDER BY. Common functions include ROW\_NUMBER(), RANK(), and DENSE\_RANK().

```
-- Get top 3 salary earners per department
WITH ranked_salaries AS (
  SELECT name, department, salary,
         DENSE_RANK() OVER (
           PARTITION BY department
           ORDER BY salary DESC
         ) as rank
  FROM employees
)
SELECT * FROM ranked_salaries WHERE rank <= 3;
```

*Interview tip: Window functions are powerful for analytical queries, reporting, and ranking without complex self-joins.*

**Q110**

★★★★★

**INTERMEDIATE**

**How do you handle soft deletes and what are the index/query implications?**

**Answer:**

Soft deleting marks a record as deleted (e.g. is\_deleted = true or deleted\_at = TIMESTAMP) rather than removing it. Query implication: every query must filter with WHERE deleted\_at IS NULL. Index implication: you should create partial/filtered indexes on active records to avoid scanning deleted rows, especially if deletes are frequent.

```
-- 1. Create table with soft delete column
ALTER TABLE users ADD COLUMN deleted_at TIMESTAMP;

-- 2. Create partial index for active users only
CREATE INDEX idx_active_users_email ON users(email) WHERE deleted_at IS NULL;

-- 3. Querying active users
```

```
SELECT * FROM users WHERE email = 'test@test.com' AND deleted_at IS NULL;
```

*Interview tip: Partial indexes keep indexes small and queries fast by ignoring soft-deleted rows entirely.*

**Q111**

★★★★★

**ADVANCED**

## Explain Database Sharding vs Partitioning.

### Answer:

Partitioning (vertical or horizontal) splits tables within a single database instance (e.g., PostgreSQL table partitioning by date range). Sharding splits data across multiple independent database instances or servers (shared-nothing architecture). Partitioning simplifies queries on large tables on a single server, while sharding scales writes and storage across multiple hardware nodes.

```
-- PostgreSQL declarative range partitioning (Horizontal Partitioning)
CREATE TABLE orders (
  id SERIAL,
  order_date DATE NOT NULL,
  amount DECIMAL
) PARTITION BY RANGE (order_date);

-- Create sub-tables
CREATE TABLE orders_2025 PARTITION OF orders
  FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
```

*Interview tip: Partition first to handle large tables. Shard only when a single database server cannot handle the throughput.*

## Q112

★★★★★

BASIC

**What is Big O notation? Give examples.**

**Answer:**

Big O describes how an algorithm scales with input size — it's the worst-case growth rate.  $O(1)$  constant (hash lookup),  $O(\log n)$  binary search,  $O(n)$  linear scan,  $O(n \log n)$  merge sort,  $O(n^2)$  nested loops,  $O(2^n)$  recursive subsets. Always state both time AND space complexity.

```
// O(1) - hash lookup
const map = new Map();
map.get('key'); // constant

// O(n) - linear scan
array.find(x => x === target);

// O(log n) - binary search
function binarySearch(arr, target) {
  let lo = 0, hi = arr.length - 1;
  while (lo <= hi) {
    const mid = (lo + hi) >> 1;
    if (arr[mid] === target) return mid;
    arr[mid] < target ? lo = mid+1 : hi = mid-1;
  }
  return -1;
}
```

*Interview tip: In interviews: state time complexity, then space complexity — both matter.*

## Q113

★★★★★

BASIC

**Two Sum problem — optimal approach.**

**Answer:**

Naive:  $O(n^2)$  nested loops. Optimal: one pass with a hash map —  $O(n)$  time,  $O(n)$  space. For each number, check if its complement (target - num) exists in the map. This 'complement lookup' pattern solves dozens of interview problems.

```
function twoSum(nums, target) {
  const map = new Map(); // value -> index

  for (let i = 0; i < nums.length; i++) {
    const complement = target - nums[i];

    if (map.has(complement)) {
      return [map.get(complement), i];
    }
    map.set(nums[i], i);
  }
  return [];
}

// twoSum([2,7,11,15], 9) -> [0,1]
// O(n) time, O(n) space
```

*Interview tip: The complement pattern in a hash map is the most common interview optimization technique.*

**Reverse a linked list.****Answer:**

Use three pointers: prev, current, next. At each step: save next, point current backward to prev, advance prev and current. After the loop, prev is the new head. This is the most frequently asked data structure question.

```
function reverseList(head) {
  let prev = null;
  let current = head;

  while (current) {
    const next = current.next; // save
    current.next = prev;      // reverse
    prev = current;           // advance
    current = next;
  }
  return prev; // new head
}

// 1->2->3->null becomes 3->2->1->null
// O(n) time, O(1) space
```

*Interview tip: Memorize the 3-pointer technique: save, reverse, advance. Works for many linked list problems.*

**Valid Parentheses — stack solution.****Answer:**

Use a stack. For each character: if opening bracket, push its expected closing bracket. If closing bracket, pop from stack and compare. If mismatch or stack not empty at end, invalid. This is the classic stack problem — demonstrates LIFO perfectly.

```
function isValid(s) {
  const stack = [];
  const map = { '(': ')', '[': ']', '{': '}' };

  for (const char of s) {
    if (map[char]) {
      stack.push(map[char]); // push expected closer
    } else {
      if (stack.pop() !== char) return false;
    }
  }
  return stack.length === 0;
}

// isValid('[]{}') -> true
// isValid('([)]') -> false
// O(n) time, O(n) space
```

*Interview tip: Push the EXPECTED closing bracket — this avoids complex if/else when comparing.*

## Q116



INTERMEDIATE

**What is the difference between vertical and horizontal scaling?****Answer:**

Vertical: add more CPU/RAM to one machine. Simple but has a ceiling and is a single point of failure. Horizontal: add more machines behind a load balancer. No ceiling, no SPOF, but requires stateless design. Modern systems prefer horizontal for reliability.

```
// Horizontal scaling requires stateless services
// Bad: session in memory (only one server has it)
app.use(session({ secret: 'x', resave: false }));

// Good: session in Redis (shared across servers)
app.use(session({
  store: new RedisStore({ client: redisClient }),
  secret: process.env.SESSION_SECRET,
  resave: false,
}));

// JWT is inherently stateless – even better
```

*Interview tip: Horizontal scaling + stateless design is the foundation of cloud-native architecture.*

## Q117



INTERMEDIATE

**Explain caching strategies: cache-aside, write-through, write-behind.****Answer:**

Cache-aside (lazy loading): check cache → miss → load from DB → populate cache. Most common. Write-through: write to cache AND DB simultaneously — consistent but slower writes. Write-behind: write to cache first, async persist to DB — fastest writes, risk of data loss on crash.

```
// Cache-aside pattern
async function getUser(id) {
  const key = 'user:' + id;
  const cached = await redis.get(key);
  if (cached) return JSON.parse(cached);

  const user = await db.findById(id);
  await redis.setex(key, 3600, JSON.stringify(user));
  return user;
}

// Invalidate on update
async function updateUser(id, data) {
  await db.update(id, data);
  await redis.del('user:' + id);
}
```

*Interview tip: Always set TTL on cached entries — never cache without expiry.*

## Q118



INTERMEDIATE

**Design a URL shortener system.****Answer:**

Requirements: shorten URLs, redirect fast, 1M+ requests/day. Components: API server, ID generator (base62 or auto-increment or hash), Redis (fast lookup), PostgreSQL (persistent store), CDN for edge redirect. Keep it simple — generate short code, store mapping, redirect on hit.

```
// ID generation (base62)
const CHARS = '0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ';
function encode(num) {
  let result = '';
  while (num > 0) {
    result = CHARS[num % 62] + result;
    num = Math.floor(num / 62);
  }
  return result.padStart(7, '0');
}

// Request flow:
// POST /shorten -> generate ID -> store in DB + Redis
// GET /:id -> Redis lookup (cache hit ~99%)
// -> 301 redirect to original URL
```

*Interview tip: Always start system design: requirements → estimation → high-level → deep dive.*

**Q119**

★★★★★

**INTERMEDIATE**

### Design a real-time chat system.

#### Answer:

Components: WebSocket server (persistent connections), Redis Pub/Sub (cross-server messaging), message storage (DB), presence tracking (online/offline). Users connect via WebSocket, messages are broadcast through Redis pub/sub to all servers, then persisted to DB asynchronously.

```
// Architecture:
// Client <-> WebSocket Server <-> Redis Pub/Sub
//                                     |
//                                     Message Queue
//                                     |
//                                     MongoDB/Postgres

// Key decisions:
// 1. WebSocket for real-time delivery
// 2. Redis pub/sub for cross-server broadcast
// 3. Message queue for async persistence
// 4. Online status: Redis SET with TTL
// 5. Message history: cursor-based pagination
// 6. Read receipts: separate events
```

*Interview tip: For 1:1 chats, route through user channels. For group chats, use room-based pub/sub.*

**Q120**

★★★★★

**INTERMEDIATE**

### Explain BFS vs DFS traversal.

#### Answer:

BFS (Breadth-First Search): explores level by level using a queue. Best for shortest path in unweighted graphs. DFS (Depth-First Search): goes deep before backtracking using a stack/recursion. Best for exhaustive search, cycle detection, topological sort.

```
// BFS – level by level (queue)
function bfs(root) {
  const queue = [root];
  while (queue.length) {
    const node = queue.shift();
    console.log(node.val);
    if (node.left) queue.push(node.left);
    if (node.right) queue.push(node.right);
  }
}
```

```
// DFS – go deep (recursion/stack)
function dfs(node) {
  if (!node) return;
  console.log(node.val); // pre-order
  dfs(node.left);
  dfs(node.right);
}
```

*Interview tip: BFS = queue = level-by-level. DFS = stack/recursion = branch-by-branch.*

**Q121**

★★★★★

**INTERMEDIATE**

## Explain the sliding window technique.

### Answer:

Sliding window maintains a 'window' of elements as you traverse an array — expand the right side to include, shrink the left side to exclude. It turns  $O(n^2)$  brute force into  $O(n)$ . Used for: max sum subarray, longest substring without repeats, minimum window containing all characters.

```
// Max sum subarray of size k
function maxSubarraySum(arr, k) {
  let windowSum = 0;
  for (let i = 0; i < k; i++) windowSum += arr[i];

  let maxSum = windowSum;
  for (let i = k; i < arr.length; i++) {
    windowSum += arr[i] - arr[i - k]; // slide
    maxSum = Math.max(maxSum, windowSum);
  }
  return maxSum;
}

// Longest substring without repeating chars
function lengthOfLongest(s) {
  const seen = new Map();
  let left = 0, max = 0;
  for (let right = 0; right < s.length; right++) {
    if (seen.has(s[right]))
      left = Math.max(left, seen.get(s[right]) + 1);
    seen.set(s[right], right);
    max = Math.max(max, right - left + 1);
  }
  return max;
}
```

*Interview tip: When you see 'subarray' or 'substring' in a problem, think sliding window first.*

**Q122**

★★★★★

**INTERMEDIATE**

## What is the circuit breaker pattern?

### Answer:

Circuit breaker prevents cascading failures when a service is down. Three states: Closed (normal, requests pass through), Open (service is down, requests fail immediately — no waiting), Half-Open (test with a few requests to see if service recovered). Like an electrical circuit breaker — stops the damage.

```
class CircuitBreaker {
  constructor(fn, { threshold = 5, timeout = 30000 } = {}) {
    this.fn = fn;
    this.failures = 0;
    this.threshold = threshold;
  }
}
```

```

    this.timeout = timeout;
    this.state = 'CLOSED';
    this.nextAttempt = 0;
  }

  async call(...args) {
    if (this.state === 'OPEN') {
      if (Date.now() < this.nextAttempt)
        throw new Error('Circuit is OPEN');
      this.state = 'HALF_OPEN';
    }
    try {
      const result = await this.fn(...args);
      this.reset();
      return result;
    } catch (err) {
      this.recordFailure();
      throw err;
    }
  }
}

```

*Interview tip: Use circuit breakers around all external service calls — databases, APIs, third-party services.*

**Q123**

★★★★★

**INTERMEDIATE**

### Merge two sorted arrays.

#### Answer:

Use two pointers, one for each array. Compare elements at both pointers, push the smaller one, advance that pointer. When one array is exhausted, append the remainder of the other. This is the merge step of merge sort.  $O(n + m)$  time,  $O(n + m)$  space.

```

function mergeSorted(a, b) {
  const result = [];
  let i = 0, j = 0;

  while (i < a.length && j < b.length) {
    if (a[i] <= b[j]) result.push(a[i++]);
    else result.push(b[j++]);
  }

  // Append remaining
  while (i < a.length) result.push(a[i++]);
  while (j < b.length) result.push(b[j++]);

  return result;
}

// mergeSorted([1,3,5], [2,4,6]) -> [1,2,3,4,5,6]

```

*Interview tip: Two-pointer on sorted data is a fundamental technique — appears in many interview problems.*

**Q124**

★★★★★

**INTERMEDIATE**

### Explain event-driven architecture.

#### Answer:

Instead of synchronous request-response, services communicate by emitting and consuming events. Event producers don't know or care about consumers. This enables loose coupling, scalability, and resilience. Events are stored in brokers (Kafka, RabbitMQ) for reliable delivery.

```
// Event-driven flow
// OrderService emits 'OrderPlaced'
// ↓ (async via message broker)
// PaymentService listens → processes payment
// InventoryService listens → reserves stock
// NotificationService listens → sends email

// Benefits:
// 1. Services are independent
// 2. Adding consumers doesn't change producer
// 3. Events can be replayed
// 4. Natural audit trail

// Kafka example:
await producer.send({
  topic: 'orders',
  messages: [{ value: JSON.stringify(order) }]
});
```

*Interview tip: Event sourcing stores events as the source of truth — you can rebuild state by replaying events.*

**Q125**

★★★★★

**ADVANCED**

## How would you design a real-time notification system?

### Answer:

Components: event producers (any service), message broker (Kafka), notification service (routes to channels), delivery channels (WebSocket for online, push/email/SMS for offline). Use fan-out pattern for broadcasting. Store preferences so users control what they receive.

```
// Producer emits event
await kafka.send({
  topic: 'notifications',
  messages: [{ value: JSON.stringify({
    type: 'ORDER_PLACED',
    userId: user.id,
    orderId: order.id
  })}]
});

// Consumer routes notifications
kafka.on('message', async msg => {
  const event = JSON.parse(msg.value);
  // WebSocket for online users
  io.to('user:' + event.userId).emit('notification', {
    type: event.type, message: 'Order confirmed!'
  });
  // Email for offline users
  if (!onlineUsers.has(event.userId))
    await emailQueue.add(event);
});
```

*Interview tip: Separate notification routing from delivery — use dedicated workers per channel.*

**Q126**

★★★★★

**ADVANCED**

## Explain the CAP theorem.

### Answer:

CAP: a distributed system can guarantee only 2 of 3 — Consistency (all nodes see same data), Availability (every request gets a response), Partition Tolerance (works despite network failures). Since network partitions are inevitable, the real choice is CP (consistent but might refuse requests) or AP (available but might serve stale data).

```
// CP systems: MongoDB, ZooKeeper, HBase
// During partition: refuse writes to stay consistent
// Use when: bank balances, inventory counts

// AP systems: Cassandra, DynamoDB, CouchDB
// During partition: serve potentially stale data
// Use when: social feeds, product catalog

// Tunable consistency example (Cassandra):
CONSISTENCY QUORUM; -- majority of nodes must agree
CONSISTENCY EVENTUAL; -- fast, may read stale
```

*Interview tip: Modern systems often have configurable consistency (e.g., MongoDB readConcern/writeConcern).*

**Q127**

★★★★★

**ADVANCED**

## Explain database connection pooling.

### Answer:

Creating a new DB connection is expensive (TCP handshake, auth, memory). Connection pooling maintains a pool of pre-established connections reused across requests. Without pooling, high traffic exhausts DB connections. pg-pool (Postgres), Mongoose (MongoDB) pool by default.

```
// PostgreSQL with pg pool
const { Pool } = require('pg');

const pool = new Pool({
  host: process.env.DB_HOST,
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  max: 20, // max connections
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
});

// Reuses existing connection from pool
const result = await pool.query(
  'SELECT * FROM users WHERE id = $1', [id]
);
```

*Interview tip: Tune max pool size based on DB max\_connections. Rule: pool size = (number of cores \* 2) + effective spindle count.*

**Q128**

★★★★★

**ADVANCED**

## Design an API gateway.

### Answer:

API gateway is the single entry point for all client requests. It handles: routing to microservices, authentication, rate limiting, request/response transformation, caching, logging, and load balancing. Clients talk to one gateway instead of many services.

```
// API Gateway responsibilities:
// 1. Route: /api/users/* -> user-service
// 2. Route: /api/orders/* -> order-service
// 3. Auth: verify JWT before forwarding
// 4. Rate limit: per user/IP
// 5. Transform: aggregate responses
// 6. Cache: store frequent reads
// 7. Circuit break: fallback on failure

// Tools: Kong, AWS API Gateway,
//        Express gateway, Nginx
```

```
// Simple Express gateway
app.use('/api/users', authMiddleware,
  proxy('http://user-service:3001'));
app.use('/api/orders', authMiddleware,
  proxy('http://order-service:3002'));
```

*Interview tip: Keep the gateway thin — business logic belongs in services, not the gateway.*

**Q129**

★★★★★

**ADVANCED**

## Explain hash table collision resolution.

### Answer:

When two keys hash to the same slot, you have a collision. Two strategies: Chaining (each slot holds a linked list of entries) — simple, grows naturally. Open Addressing (probe for next empty slot: linear, quadratic, double hashing) — better cache performance, fixed size.

```
// Chaining: slot holds a list
// Hash table:
// [0] -> null
// [1] -> ('alice', 25) -> ('bob', 30)
// [2] -> ('charlie', 22)

// Simple hash map with chaining
class HashMap {
  constructor(size = 53) {
    this.table = new Array(size);
  }
  hash(key) {
    let total = 0;
    for (let i = 0; i < key.length; i++)
      total = (total * 31 + key.charCodeAt(i)) % this.table.length;
    return total;
  }
  set(key, val) {
    const idx = this.hash(key);
    if (!this.table[idx]) this.table[idx] = [];
    this.table[idx].push([key, val]);
  }
}
```

*Interview tip: JavaScript Map uses hash tables internally —  $O(1)$  average for get/set/delete.*

**Q130**

★★★★★

**ADVANCED**

## How do you design a distributed Rate Limiter?

### Answer:

Four common algorithms: 1) Token Bucket: bucket holds max tokens, refilled at a rate. Allows bursts. 2) Leaky Bucket: queue processed at a constant rate. Smooths traffic. 3) Sliding Window Log: store timestamps of requests in Redis ZSET. 4) Sliding Window Counter: compute rate combining current and previous window counts (low memory).

```
// Sliding Window Counter conceptual implementation in Redis
async function isRateLimited(userId, limit, windowSizeSecs) {
  const now = Date.now();
  const clearBefore = now - (windowSizeSecs * 1000);
  const key = `rate:${userId}`;
  const tx = redis.multi();
  tx.zremrangebyscore(key, 0, clearBefore);
  tx.zcard(key);
  tx.zadd(key, now, now.toString());
}
```

```
tx.expire(key, windowSizeSecs);
const results = await tx.exec();
return results[1][1] >= limit;
}
```

*Interview tip: Use Redis for distributed rate limiting to share the state across multiple application servers.*

**Q131**

★★★★★

**ADVANCED**

## Design a Content Delivery Network (CDN) and explain how edge caching works.

### Answer:

A CDN is a distributed network of proxy servers deployed globally. Routing (Anycast or GeoDNS) directs clients to the closest edge server. Edge caching stores files locally. If a cache miss occurs, the edge fetches from the origin, caches it, and returns it. Uses HTTP caching headers like Cache-Control.

```
// Caching headers sent by origin:
// Cache-Control: public, max-age=31536000, s-maxage=86400
// - max-age: browser caches for 1 year
// - s-maxage: CDN edge caches for 1 day

// Purge endpoint to invalidate cache programmatically:
// POST /api/v1/purge { files: ['/logo.png'] }
// Purging is critical for immediate updates.
```

*Interview tip: CDNs are critical for lowering Time to First Byte (TTFB) and reducing load on origin servers.*

**Q132**

★★★★★

**ADVANCED**

## Explain the Consistent Hashing algorithm and its role in distributed systems.

### Answer:

Traditional hashing ( $\text{hash}(\text{key}) \% N$ ) reshuffles almost all keys when  $N$  (number of servers) changes. Consistent Hashing maps servers and keys to a 360-degree circular ring. A key routes to the first server clockwise. Adding/removing a server only remaps  $1/N$  of the keys. Virtual nodes (vnodes) distribute keys evenly.

```
// Hash space: [0, 2^32 - 1]
// Server A: hash = 100,000
// Server B: hash = 2,000,000
// Key X: hash = 500,000 -> Routes clockwise to Server B
// If Server A goes offline, only keys on Server A remap to B.
// Keys on Server B are unaffected.
```

*Interview tip: Consistent Hashing is used in DynamoDB, Cassandra, Memcached, and load balancers.*

**Q133**

★★★★★

**ADVANCED**

## Design a distributed unique ID generator (like Twitter Snowflake).

### Answer:

Requirements: unique, time-sortable, 64-bit integer. Twitter Snowflake allocates: 1 bit (unused), 41 bits (timestamp in ms, 69 years lifetime), 10 bits (machine/datacenter ID, 1024 nodes), and 12 bits (sequence number, auto-incremented for IDs generated in the same ms, 4096 IDs/ms).

```
// Layout: | 1-bit unused | 41-bit time | 10-bit node | 12-bit seq |
const EPOCH = 1600000000000n;

function generateId(nodeId, seq) {
  const timestamp = BigInt(Date.now()) - EPOCH;
  return (timestamp << 22n) |
    (BigInt(nodeId) << 12n) |
    BigInt(seq);
}
```

```
}
```

*Interview tip: Snowflake IDs are 64-bit sortable integers, which are highly index-friendly for databases.*

### Q134



INTERMEDIATE

**Find the longest substring without repeating characters.**

**Answer:**

Use the sliding window technique with two pointers (left and right) and a Map to track character indices. As right expands, if a character is repeated, move left to  $\max(\text{left}, \text{lastSeenIndex} + 1)$  to exclude the repeat. This runs in  $O(n)$  time and  $O(\min(a, b))$  space.

```
function longestUniqueSubstring(s) {
  let maxLen = 0, left = 0;
  const seen = new Map(); // char -> index
  for (let right = 0; right < s.length; right++) {
    const char = s[right];
    if (seen.has(char)) {
      left = Math.max(left, seen.get(char) + 1);
    }
    seen.set(char, right);
    maxLen = Math.max(maxLen, right - left + 1);
  }
  return maxLen;
}
// longestUniqueSubstring('abcabcbb') -> 3
```

*Interview tip: Sliding window reduces what would be an  $O(n^2)$  or  $O(n^3)$  nested search to a clean  $O(n)$  linear pass.*

### Q135



ADVANCED

**Design a File Upload system with multipart upload and resume capability.**

**Answer:**

For large files, uploading in one request is fragile. Design: 1) Client initiates upload: server returns uploadId. 2) Client splits file into chunks (e.g. 5MB) and uploads them in parallel using presigned URLs with part numbers. 3) Server stores chunks in temp bucket. 4) Client completes upload: server merges chunks and verifies checksum.

```
// 1. Initiate upload
// POST /api/upload/initiate -> { uploadId: 'id', chunkUrls: [...] }

// 2. Upload chunk in parallel
// PUT /chunk-url-1?partNumber=1&uploadId=id -> ETag: 'etag1'

// 3. Complete upload
// POST /api/upload/complete -> { parts: [{ ETag: 'etag1', PartNumber: 1 }] }
```

*Interview tip: Multipart upload enables automatic retries of failed parts and parallel upload speedups.*

### Q136



ADVANCED

**Explain LRU (Least Recently Used) Cache design and its implementation details.**

**Answer:**

An LRU cache evicts the least recently accessed item when capacity is reached. Requirements: get and put operations must run in  $O(1)$  time. Implementation: use a Hash Map for  $O(1)$  key-to-node lookups, and a Doubly Linked List (DLL) to maintain access order. Accessed items move to the head, evictions happen at the tail.

```
class Node {
  constructor(key, val) {
    this.key = key; this.val = val;
    this.prev = null; this.next = null;
  }
}
// HashMap contains references to DLL Nodes.
// get(key): retrieve from map, move to DLL head, return value.
// put(key, val): if exists, update + move to head. If new, add to head.
//               if over capacity, delete DLL tail and remove from map.
```

*Interview tip: Using a DLL allows  $O(1)$  node removal and insertion, while a Map provides  $O(1)$  lookup.*

---

## Phase 6: Scalable Architecture & Production DevOps

25 Q&As

Q137

★★★★★

BASIC

**What is the difference between monolithic and microservices architecture?**

**Answer:**

Monolith: single deployable unit, all features in one codebase — simple to develop, test, and deploy. Microservices: independently deployable services, each owning its domain and database — enables independent scaling and team autonomy but adds operational complexity (networking, monitoring, debugging).

```
// Monolith – one deployment
app/
  src/
    auth/
    users/
    products/
    payments/

// Microservices – many deployments
auth-service/    -> :3001
user-service/    -> :3002
product-service/ -> :3003
payment-service/ -> :3004
// Each communicates via HTTP/gRPC/events
// Each has its own database
```

*Interview tip: Most startups: start monolith, extract microservices when specific pain points appear.*

Q138

★★★★★

BASIC

**What is Docker and why is it used?**

**Answer:**

Docker packages your app + all dependencies into a container — a portable unit that runs identically everywhere. 'Works on my machine' becomes 'works everywhere'. Containers share the host OS kernel (unlike VMs), start in seconds, and use minimal resources.

```
# Multi-stage Dockerfile for Next.js
FROM node:20-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci

FROM deps AS builder
COPY . .
RUN npm run build

FROM node:20-alpine AS runner
WORKDIR /app
COPY --from=builder /app/.next ./.next
COPY --from=builder /app/public ./public
COPY --from=deps /app/node_modules ./node_modules
EXPOSE 3000
CMD ["npm", "start"]
```

*Interview tip: Multi-stage builds keep images small by excluding build tools from the final image.*

## What is CI/CD and why does it matter?

### Answer:

CI (Continuous Integration): automatically test and build on every push — catch bugs before merge. CD (Continuous Deployment): automatically deploy passing builds to production. Together: push code → tests run → build → deploy. No manual steps, no human error.

```
# .github/workflows/deploy.yml
name: CI/CD Pipeline
on:
  push:
    branches: [main]
jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - run: npm test
      - run: npm run build
      - name: Deploy
        run: |
          ssh user@server 'cd /app &&
            git pull && npm ci &&
            npm run build &&
            pm2 restart all'
```

*Interview tip: Never deploy to production without automated tests passing in CI.*

## What is load balancing? Explain load balancing algorithms.

### Answer:

Load balancing distributes requests across multiple servers to prevent overload. Algorithms: Round Robin (rotate equally), Least Connections (route to least busy), IP Hash (consistent routing per user), Weighted (more traffic to stronger servers).

```
# Nginx load balancer config
upstream backend {
    least_conn;

    server app1.example.com:3000 weight=3;
    server app2.example.com:3000 weight=2;
    server app3.example.com:3000 weight=1;

    keepalive 32;
}

server {
    listen 80;
    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_connect_timeout 5s;
        proxy_read_timeout 30s;
    }
}
```

Q141

★★★★★

INTERMEDIATE

### How does Redis work as a cache? Explain eviction policies.

#### Answer:

Redis is an in-memory key-value store with sub-millisecond latency. When memory fills up, eviction policies decide what to remove. allkeys-lru (Least Recently Used) is the standard cache policy — evicts least-used keys regardless of TTL. volatile-lru only evicts keys that have a TTL set.

```
const redis = require('ioredis');
const client = new redis(process.env.REDIS_URL);

// Set with 1-hour TTL
await client.setex('product:123', 3600,
  JSON.stringify(product));

// Get
const raw = await client.get('product:123');
if (raw) return JSON.parse(raw);

// Atomic operations
await client.incr('page:views:home');
await client.lpush('recent:products', productId);
await client.ltrim('recent:products', 0, 9); // keep 10
```

Interview tip: Set maxmemory and maxmemory-policy in redis.conf. allkeys-lru is the standard cache policy.

Q142

★★★★★

INTERMEDIATE

### What are message queues and when should you use them?

#### Answer:

Message queues decouple producers from consumers for async processing. Use when: tasks are slow (email, video encoding), you need retry logic, you need to absorb traffic spikes, or services should be loosely coupled. The queue buffers work and guarantees delivery.

```
const { Queue, Worker } = require('bullmq');

// Producer – add to queue
const emailQueue = new Queue('email', { connection: redis });

await emailQueue.add('welcome', { userId }, {
  attempts: 3,
  backoff: { type: 'exponential', delay: 1000 }
});

// Consumer – process jobs
new Worker('email', async job => {
  const { userId } = job.data;
  const user = await db.findById(userId);
  await mailer.sendWelcome(user.email);
}, { connection: redis });
```

Interview tip: BullMQ (Redis-based) is great for Node.js. For massive event streaming, use Apache Kafka.

### Explain the production infrastructure flow for a scalable app.

**Answer:**

Request journey: User → CDN (edge cache) → Load Balancer → App Servers (stateless) → Cache Layer (Redis) → Database (primary writes / replica reads). Background jobs flow through message queues to workers. Each layer can scale independently.

```
// Production flow:
// 1. User hits CDN (Cloudflare/CloudFront)
//    -> cached: serve immediately
//    -> miss: forward to origin

// 2. Load Balancer (Nginx/AWS ALB)
//    -> distributes to app server pool

// 3. App Server (stateless Next.js/Node)
//    -> check Redis cache
//    -> cache hit: return immediately
//    -> cache miss: query PostgreSQL replica
//    -> write operations: PostgreSQL primary

// 4. Background tasks:
//    -> BullMQ/Kafka -> Workers
//    -> (email, notifications, analytics)
```

*Interview tip: Always make app servers stateless — store sessions, cache, and feature flags in Redis.*

### Explain blue-green vs canary deployments.

**Answer:**

Blue-green: two identical environments — Blue (current) and Green (new). Deploy to Green, test it, then switch traffic instantly. Instant rollback by switching back. Canary: gradually route a small percentage of traffic (1%, 5%, 25%...) to the new version. Monitor metrics at each step. Roll back if errors spike.

```
// Blue-green deployment
// 1. Deploy new version to 'green' servers
// 2. Run smoke tests on green
// 3. Switch load balancer to green
// 4. Old 'blue' becomes standby
// Rollback: switch LB back to blue

// Canary deployment
// 1. Deploy new version to 1 server
// 2. Route 5% traffic to canary
// 3. Monitor error rate, latency, p99
// 4. If healthy: increase to 25% -> 50% -> 100%
// 5. If unhealthy: route 100% back to old

// Nginx canary
upstream backend {
    server old-v1:3000 weight=95;
    server new-v2:3000 weight=5;
}
```

*Interview tip: Blue-green is simpler. Canary is safer for large-scale systems with millions of users.*

## What is serverless architecture?

### Answer:

Serverless means you write functions, the cloud runs them — no server management. You pay only for execution time (not idle servers). AWS Lambda, Vercel Edge Functions, Cloudflare Workers. Trade-offs: cold starts, execution time limits, vendor lock-in, harder debugging.

```
// AWS Lambda function
export const handler = async (event) => {
  const { userId } = JSON.parse(event.body);
  const user = await db.findUser(userId);
  return {
    statusCode: 200,
    body: JSON.stringify(user),
  };
};

// Vercel Edge Function
export const config = { runtime: 'edge' };
export default function handler(req) {
  return new Response(JSON.stringify({ hello: 'world' }), {
    headers: { 'Content-Type': 'application/json' },
  });
}
```

*Interview tip: Serverless is great for: APIs, webhooks, cron jobs, image processing. Bad for: long-running tasks, WebSockets.*

## Explain feature flags and progressive rollouts.

### Answer:

Feature flags let you deploy code with features hidden behind toggles — enable for specific users, percentages, or environments without redeploying. Progressive rollout: enable for internal team → beta users → 10% → 50% → 100%. If something breaks, disable the flag instantly.

```
// Simple feature flag
if (featureFlags.isEnabled('new-checkout', user)) {
  return <NewCheckout />;
} else {
  return <OldCheckout />;
}

// With LaunchDarkly / Unleash
const showFeature = ldClient.variation(
  'new-checkout',
  { key: user.id, email: user.email },
  false // default
);

// Rollout rules:
// - Internal team: 100%
// - Beta users: 100%
// - Production: 10% (gradually increase)
```

*Interview tip: Feature flags decouple deployment from release — deploy daily, release when ready.*

## What is Kubernetes and how does it orchestrate containers?

### Answer:

Kubernetes (K8s) manages containers at scale: scheduling across nodes, self-healing (restarts failed pods), rolling deployments (zero downtime), HPA (auto-scaling based on metrics), service discovery, load balancing, and secret management. It's the industry standard for container orchestration.

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 3
  selector:
    matchLabels: { app: api }
  template:
    spec:
      containers:
        - name: api
          image: myregistry/api:v1.2.3
          resources:
            requests: { cpu: 100m, memory: 128Mi }
            limits: { cpu: 500m, memory: 512Mi }
---
# Auto-scale 3-20 pods based on CPU
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

*Interview tip: Start with Docker Compose for local dev. Graduate to Kubernetes when managing 5+ services.*

## How would you design a system for 1 million concurrent users?

### Answer:

Layer the architecture: CDN → Load Balancer → Stateless App Servers (auto-scaling) → Redis Cache Cluster → DB (primary + read replicas) → Message Queues → Background Workers → Monitoring. Key: estimate capacity first, then design each layer to handle it.

```
// Capacity estimation for 1M users:
// Peak: ~50K req/sec (assume 5% concurrent)
// Avg response: 100ms -> 5000 RPS per server
// -> Need ~10 app servers at peak
// DB reads: 80% -> route to 3 replicas
// DB writes: 20% -> primary
// Cache hit rate target: 95%+

// Auto-scaling rule:
// Scale out when CPU > 70% for 2 minutes
// Scale in when CPU < 30% for 5 minutes
// Min: 3 instances (HA)
```

```
// Max: 50 instances (cost cap)
```

*Interview tip: Always estimate: traffic → server count → DB connections → cache hit rate → infrastructure cost.*

## Q149

★★★★★

ADVANCED

### Explain distributed tracing and observability.

#### Answer:

Observability has 3 pillars: Metrics (numbers — Prometheus/Grafana), Logs (text — centralized logging), Traces (request journeys — distributed tracing). Distributed tracing tracks a request across services using a shared trace ID. Use OpenTelemetry as the standard instrumentation.

```
// OpenTelemetry setup
const { trace } = require('@opentelemetry/api');

async function processOrder(orderId) {
  const tracer = trace.getTracer('order-service');
  const span = tracer.startSpan('processOrder');
  span.setAttribute('order.id', orderId);

  try {
    const order = await db.findOrder(orderId);
    span.addEvent('order.fetched');

    await paymentService.charge(order);
    span.setStatus({ code: SpanStatusCode.OK });
  } catch (err) {
    span.recordException(err);
    span.setStatus({ code: SpanStatusCode.ERROR });
    throw err;
  } finally {
    span.end();
  }
}
```

*Interview tip: Key SLIs to monitor: latency (p50/p95/p99), error rate (%), throughput (RPS), saturation (CPU/mem).*

## Q150

★★★★★

ADVANCED

### What is the Saga pattern in microservices?

#### Answer:

Saga manages distributed transactions across microservices — each service has its own database, so no global SQL transactions. Two flavors: Choreography (services emit events that trigger next steps) vs Orchestration (a central coordinator calls each service). On failure, compensating transactions undo completed steps.

```
// Orchestration Saga
class OrderSaga {
  async execute(orderId) {
    try {
      await inventoryService.reserve(orderId);
      await paymentService.charge(orderId);
      await shippingService.schedule(orderId);
      await notifyService.send(orderId);
    } catch (err) {
      // Compensate — rollback in reverse
      await shippingService.cancel(orderId);
      await paymentService.refund(orderId);
      await inventoryService.release(orderId);
      throw err;
    }
  }
}
```

```
}  
}
```

*Interview tip: Compensating transactions are the microservices equivalent of SQL ROLLBACK.*

**Q151**

★★★★★

**ADVANCED**

## What is Infrastructure as Code (IaC)?

### Answer:

IaC manages infrastructure (servers, networks, databases) through code files instead of manual configuration. Benefits: version controlled, reproducible, reviewable, automated. Terraform is cloud-agnostic. Pulumi uses real programming languages. AWS CDK is AWS-specific.

```
# Terraform – provision AWS resources  
resource "aws_instance" "api_server" {  
  ami           = "ami-0c55b159cbfafa1f0"  
  instance_type = "t3.medium"  
  tags = { Name = "api-server" }  
}  
  
resource "aws_rds_instance" "postgres" {  
  engine           = "postgres"  
  engine_version   = "15.4"  
  instance_class    = "db.t3.medium"  
  allocated_storage = 100  
  multi_az          = true  
}  
  
# Apply changes  
# terraform plan (preview)  
# terraform apply (execute)
```

*Interview tip: Never configure production infrastructure manually — use IaC for everything.*

**Q152**

★★★★★

**ADVANCED**

## Explain secrets management in production.

### Answer:

Never store secrets in code, env files, or git. Use dedicated secret managers: AWS Secrets Manager, HashiCorp Vault, Google Secret Manager. They provide encryption, access control, rotation, and audit logging. Rotate secrets regularly and grant least-privilege access.

```
// AWS Secrets Manager  
const { SecretsManager } = require('@aws-sdk/client-secrets-manager');  
const client = new SecretsManager({ region: 'us-east-1' });  
  
async function getSecret(name) {  
  const response = await client.getSecretValue({  
    SecretId: name  
  });  
  return JSON.parse(response.SecretString);  
}  
  
const dbCreds = await getSecret('prod/database');  
// { host, port, username, password }  
  
// Kubernetes secrets  
// kubectl create secret generic db-creds \  
//   --from-literal=password=xxx
```

*Interview tip: Rotate secrets regularly. If a secret is exposed, rotate ALL related credentials immediately.*

## What are common production engineering mistakes to avoid?

### Answer:

7 deadly sins: 1) No monitoring — blind when things break. 2) Hardcoded secrets — security breach waiting to happen. 3) No rate limiting — open to brute force. 4) Unhandled promise rejections — crash Node processes. 5) No backups — catastrophic data loss. 6) No CI/CD — error-prone manual deploys. 7) Premature microservices.

```
// Handle all unhandled rejections
process.on('unhandledRejection', (reason, promise) => {
  logger.error('Unhandled rejection:', reason);
  // Graceful shutdown
  server.close(() => process.exit(1));
});

// Validate env vars on startup
const required = ['DB_URL', 'JWT_SECRET', 'REDIS_URL'];
for (const key of required) {
  if (!process.env[key])
    throw new Error(key + ' env var not set');
}

// Graceful shutdown
process.on('SIGTERM', async () => {
  await server.close();
  await db.end();
  process.exit(0);
});
```

*Interview tip: Add a production readiness checklist: monitoring, logging, backups, secrets, rate limits, graceful shutdown.*

## Explain the Twelve-Factor App methodology.

### Answer:

A methodology for building modern, scalable, cloud-native SaaS applications. Key factors: 1) Codebase: one repo, many deploys. 2) Dependencies: explicitly declare/isolate. 3) Config: store in environment. 4) Backing services: treat as attached resources. 6) Processes: stateless, share-nothing. 9) Disposability: fast startup and graceful shutdown. 11) Logs: treat as streams.

```
// Factor 3: Config in env (good)
const dbUrl = process.env.DATABASE_URL;

// Factor 9: Disposability (graceful shutdown on SIGTERM)
process.on('SIGTERM', () => {
  server.close(() => {
    db.disconnect();
    process.exit(0);
  });
});
```

*Interview tip: Adhering to the Twelve-Factor App methodology ensures portability, scalability, and ease of deployment in containerized environments.*

## Q155



ADVANCED

### What is GitOps and how does it differ from traditional CI/CD?

#### Answer:

GitOps is an operational framework where Git is the single source of truth for declarative infrastructure and applications. In traditional CI/CD, the CI pipeline runs a script to push changes to the cluster (push-based). In GitOps, an agent running inside the cluster (like ArgoCD or Flux) continuously pulls the declarative state from Git and reconciles differences (pull-based). This is more secure and prevents configuration drift.

```
# gitops/argo-app.yaml (ArgoCD Application Definition)
apiVersion: argoproj.io/v1alpha1
kind: Application
spec:
  source:
    repoURL: 'https://github.com/org/infra-gitops.git'
    path: k8s/production
  destination:
    server: 'https://kubernetes.default.svc'
    namespace: default
  syncPolicy:
    automated: { prune: true, selfHeal: true }
```

*Interview tip: GitOps keeps your cluster configuration sync automatic, self-healing, and fully auditable via git commit logs.*

## Q156



ADVANCED

### Explain high availability (HA) and multi-region active-active deployments.

#### Answer:

HA ensures a system remains operational with minimal downtime (aiming for 99.99% 'four nines'). Multi-region active-active deployment means running identical fully-functioning application stack instances in multiple geographical locations (regions) simultaneously, routing traffic using latency-based DNS. This protects against entire cloud region failures and provides low-latency access globally. It requires a distributed, replicated database.

```
// Route 53 latency routing JSON config (conceptual DNS record)
{
  "Name": "api.myapp.com",
  "Type": "A",
  "SetIdentifier": "us-east-instance",
  "Region": "us-east-1",
  "AliasTarget": { "DNSName": "us-elb.myapp.com" }
}
// Route 53 handles routing based on client round-trip-time dynamically.
```

*Interview tip: Active-active requires databases designed for distributed writes (like CockroachDB, Spanner, or DynamoDB with Global Tables).*

## Q157



INTERMEDIATE

### Explain the differences between gRPC and REST/GraphQL.

#### Answer:

REST and GraphQL use JSON over HTTP/1.1 (human-readable, text-based, high overhead). gRPC (Google Remote Procedure Call) uses Protocol Buffers (Protobuf) over HTTP/2 (binary-serialized, highly compact, multiplexed connections). gRPC is schema-first (defined in .proto files) and supports bidirectional streaming. It is ideal for internal microservices communication, but harder to consume from web browsers directly.

```
// user.proto
syntax = "proto3";

message UserRequest { string id = 1; }
message UserResponse { string id = 1; string name = 2; }

service UserService {
  rpc GetUser (UserRequest) returns (UserResponse);
}
```

*Interview tip: Use gRPC for fast, type-safe communication between internal microservices. Use REST or GraphQL for client-facing public APIs.*

**Q158**

★★★★★

**ADVANCED**

## What is Service Mesh (e.g. Istio) and when do you need it?

### Answer:

A service mesh is a dedicated infrastructure layer for handling service-to-service communication in microservices. It deploys a proxy (like Envoy) as a 'sidecar' next to every service container. The sidecars intercept all network traffic, providing: mutual TLS (mTLS) encryption automatically, traffic splitting (canary routing), rate limiting, distributed tracing injection, and circuit breaking without modifying application code.

```
# Istio VirtualService for canary deployment (traffic routing)
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
spec:
  hosts: [ "payment-service" ]
  http:
    - route:
      - destination: { host: "payment-service", subset: "v1" }
        weight: 90
      - destination: { host: "payment-service", subset: "v2" }
        weight: 10
```

*Interview tip: A service mesh is usually overkill for small systems, but becomes critical as you scale to dozens of microservices.*

**Q159**

★★★★★

**ADVANCED**

## Explain CDN Edge Computing vs traditional serverless functions.

### Answer:

Traditional serverless (AWS Lambda) runs in a single region, spins up a micro-VM (cold start up to seconds), and supports full Node.js runtime. Edge computing (Vercel Edge, Cloudflare Workers) runs inside CDN edge nodes globally, starts instantly (zero cold start using V8 isolates instead of VMs), but uses a subset of standard Web APIs (no `node:fs` or `child_process`). Edge is perfect for low-latency tasks like redirects, headers, or streaming AI.

```
// Edge function (Vercel Edge Runtime / Web APIs)
export const config = { runtime: 'edge' };

export default async function handler(req) {
  const geo = req.geo?.country || 'US';
  if (geo === 'BD') {
    return Response.redirect(new URL('/bd', req.url));
  }
  return new Response(JSON.stringify({ hello: 'world' }), {
    headers: { 'Content-Type': 'application/json' }
  });
}
```

*Interview tip: Use the Edge Runtime for dynamic features that need near-zero latency, like localized routing or dynamic personalization.*

---

**Q160**

★★★★★

**ADVANCED**

### How do you handle log aggregation at scale?

**Answer:**

Do not write log files to server disks — logs should be treated as continuous streams written to stdout/stderr. An agent (FluentBit, Vector, Logstash) running on the host/pod intercepts these streams, parses the JSON structure, buffers them, and ships them to a centralized indexing cluster (Elasticsearch/OpenSearch, Grafana Loki, Datadog) where they can be queried.

```
# Vector configuration snippet (collecting stdout & sending to Loki)
sources:
  docker_logs:
    type: docker_logs
sinks:
  loki_output:
    type: loki
    inputs: [ "docker_logs" ]
    endpoint: "http://loki:3100"
    labels: { environment: "production", service: "{ container_name }" }
```

*Interview tip: Structured logging (writing logs as JSON objects) is critical for easy indexing and querying in aggregation tools.*

---

**Q161**

★★★★★

**INTERMEDIATE**

### Explain disaster recovery strategies: RTO and RPO.

**Answer:**

Two key metrics in Disaster Recovery (DR) planning: RTO (Recovery Time Objective) — how fast you must recover after an outage (e.g. system must be back online within 4 hours). RPO (Recovery Point Objective) — how much data you can afford to lose, measured in time (e.g. RPO = 24 hours means you restore from yesterday's backup and lose up to 24 hours of writes). Lower RTO/RPO requires active database replication and failover.

```
DR Tiering Strategies:
- Backup & Restore: RTO = hours/days, RPO = 24 hours (cheap)
- Pilot Light (DB replicated, core idle): RTO = minutes, RPO = minutes
- Warm Standby (scaled-down active cluster): RTO = minutes, RPO = seconds
- Multi-Site Active-Active: RTO = near-zero, RPO = zero (expensive)
```

*Interview tip: Test your disaster recovery plans and automated backups regularly. An untested recovery plan is not a recovery plan.*

---

## Phase 7: Testing & Quality Assurance

20 Q&As

Q162

★★★★★

BASIC

### What is the difference between Unit, Integration, and E2E testing?

#### Answer:

Unit: tests a single function/component in isolation (fast, many). Integration: tests how multiple units work together (medium speed). E2E: simulates a real user in a real browser (slow, few). The Testing Pyramid: many unit tests at the base, fewer integration in the middle, fewest E2E at the top.

```
// Unit
expect(sum(1, 2)).toBe(3);

// Integration
render(<LoginForm />);
userEvent.type(screen.getByLabelText('Email'), 'test@test.com');

// E2E
await page.goto('/login');
await page.fill('#email', 'test@test.com');
```

*Interview tip: The Testing Pyramid suggests many Unit tests, some Integration tests, and fewer E2E tests (as they are slow and brittle).*

Q163

★★★★★

BASIC

### What is Test-Driven Development (TDD)?

#### Answer:

TDD is a 3-step cycle: Red (write a failing test first) → Green (write minimum code to pass) → Refactor (improve code while keeping tests green). It forces you to think about the API before implementation. Result: better design, fewer bugs, higher confidence.

```
// 1. RED – write failing test
test('adds two numbers', () => {
  expect(add(2, 3)).toBe(5);
});
// RUN: FAIL (add doesn't exist)

// 2. GREEN – minimum code to pass
function add(a, b) { return a + b; }
// RUN: PASS

// 3. REFACTOR – improve if needed
// (already clean, move on to next test)

// 4. Next test: edge cases
test('handles negative numbers', () => {
  expect(add(-1, -2)).toBe(-3);
});
```

*Interview tip: TDD isn't about testing — it's about design. Tests drive you to write modular, decoupled code.*

## What is code coverage and what's a good target?

### Answer:

Code coverage measures what percentage of your code is executed by tests. Types: line coverage, branch coverage (if/else), function coverage. 80% is a good practical target. 100% is usually not worth the effort — diminishing returns. Focus on critical path coverage, not vanity numbers.

```
// Jest coverage report
// npx jest --coverage

// Output:
// File      | Stmts | Branch | Funcs | Lines
// -----|-----|-----|-----|-----
// user.ts   | 95%   | 88%   | 100%  | 95%
// auth.ts   | 82%   | 75%   | 90%   | 82%
// utils.ts  | 100%  | 100%  | 100%  | 100%

// jest.config.js
coverageThreshold: {
  global: {
    branches: 80,
    functions: 80,
    lines: 80,
    statements: 80,
  }
}
```

*Interview tip: High coverage ≠ good tests. You can have 100% coverage with zero meaningful assertions.*

## Explain React Testing Library best practices.

### Answer:

RTL tests behavior, not implementation. Query priority: `getByRole` > `getByLabelText` > `getByPlaceholderText` > `getByText` > `getByTestId`. Test what users see and do. Never test state directly — test the rendered output. Avoid testing implementation details that might change during refactoring.

```
// Good – testing behavior
render(<Counter />);
const button = screen.getByRole('button', { name: /increment/i });
fireEvent.click(button);
expect(screen.getByText('Count: 1')).toBeInTheDocument();

// Bad – testing implementation
wrapper.setState({ count: 1 });
expect(wrapper.state('count')).toBe(1);

// Testing async
render(<UserProfile userId='1' />);
const name = await screen.findByText('Nazmul');
expect(name).toBeInTheDocument();
```

*Interview tip: Query by accessibility roles first (getByRole). It ensures your app is accessible too.*

## How do you mock API calls in tests?

### Answer:

Use MSW (Mock Service Worker) to intercept network requests at the network level — works for both tests and local development. Unlike jest.mock, MSW doesn't require you to know which module makes the request. It mocks the network, not the code.

```
import { http, HttpResponse } from 'msw';
import { setupServer } from 'msw/node';

const server = setupServer(
  http.get('/api/user', () => {
    return HttpResponse.json({ name: 'Nazmul' });
  }),
  http.post('/api/login', async ({ request }) => {
    const body = await request.json();
    if (body.password === 'correct')
      return HttpResponse.json({ token: 'abc' });
    return HttpResponse.json(
      { error: 'Invalid' }, { status: 401 }
    );
  })
);

beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());
afterAll(() => server.close());
```

*Interview tip: MSW v2 is the industry standard for mocking APIs. It works with fetch, axios, and any HTTP client.*

## Compare Playwright vs Cypress for E2E testing.

### Answer:

Cypress: runs in the browser, excellent DX, real-time reloading, time-travel debugging. Single browser tab limitation. Playwright: runs outside the browser (CDP/WebSocket), supports multiple tabs/browsers/contexts, better for complex flows. Playwright is more powerful; Cypress has better DX for simple flows.

```
// Cypress
cy.visit('/login');
cy.get('[data-testid=email]').type('test@test.com');
cy.get('[data-testid=password]').type('password');
cy.get('button[type=submit]').click();
cy.url().should('include', '/dashboard');

// Playwright
await page.goto('/login');
await page.fill('[data-testid=email]', 'test@test.com');
await page.fill('[data-testid=password]', 'password');
await page.click('button[type=submit]');
await expect(page).toHaveURL(/dashboard/);
```

*Interview tip: Playwright for complex multi-page/multi-browser tests. Cypress for simpler, developer-focused E2E.*

## How do you test React hooks?

### Answer:

Use `@testing-library/react`'s `renderHook` to test custom hooks in isolation. It renders the hook inside a test component and gives you the result. For hooks that need providers (context, Redux), wrap with a custom wrapper.

```
import { renderHook, act } from '@testing-library/react';
import { useCounter } from './useCounter';

test('increments counter', () => {
  const { result } = renderHook(() => useCounter(0));

  expect(result.current.count).toBe(0);

  act(() => {
    result.current.increment();
  });

  expect(result.current.count).toBe(1);
});

// With provider wrapper
const wrapper = ({ children }) => (
  <AuthProvider>{children}</AuthProvider>
);
const { result } = renderHook(() => useAuth(), { wrapper });
```

*Interview tip: Always wrap state updates in `act()` — it ensures React processes all updates before assertions.*

## How do you test async code in Jest?

### Answer:

Three approaches: 1) Return a promise. 2) Use `async/await` (most common). 3) Use `done` callback (legacy). For timers, use `jest.useFakeTimers()`. Always handle both success and error paths. Use `waitFor()` in RTL for async DOM updates.

```
// async/await (preferred)
test('fetches user data', async () => {
  const user = await fetchUser(1);
  expect(user.name).toBe('Nazmul');
});

// Testing errors
test('throws on invalid id', async () => {
  await expect(fetchUser(-1))
    .rejects.toThrow('Invalid ID');
});

// Fake timers
test('debounce waits before calling', () => {
  jest.useFakeTimers();
  const fn = jest.fn();
  const debounced = debounce(fn, 500);

  debounced();
  expect(fn).not.toHaveBeenCalled();

  jest.advanceTimersByTime(500);
});
```

```
expect(fn).toHaveBeenCalledTimes(1);
});
```

*Interview tip: `jest.useFakeTimers()` is essential for testing `debounce`, `throttle`, and `setTimeout` logic.*

**Q170**

★★★★★

INTERMEDIATE

## What is snapshot testing?

### Answer:

Snapshot testing captures the rendered output of a component and saves it to a file. On subsequent runs, it compares the current output against the saved snapshot. If they differ, the test fails — you decide if the change is intentional (update snapshot) or a bug (fix code).

```
// Component snapshot
test('renders correctly', () => {
  const tree = renderer.create(
    <Button label='Click me' />
  ).toJSON();
  expect(tree).toMatchSnapshot();
});

// Inline snapshot (stored in test file)
test('renders greeting', () => {
  const { container } = render(<Greeting name='Ali' />);
  expect(container).toMatchInlineSnapshot(`
    <div>
      <h1>Hello, Ali!</h1>
    </div>
  `);
});

// Update snapshots: npx jest --updateSnapshot
```

*Interview tip: Snapshots catch unintended UI changes but can be noisy. Use sparingly for stable components.*

**Q171**

★★★★★

INTERMEDIATE

## Explain contract testing for APIs.

### Answer:

Contract testing verifies that a producer (API) and consumer (frontend/service) agree on the request/response format. Unlike integration tests, each side tests independently against a shared contract. Tools: Pact, MSW. Prevents: 'the API changed and nobody told the frontend'.

```
// Pact contract test (consumer side)
const interaction = {
  state: 'user exists',
  uponReceiving: 'a request for user 1',
  withRequest: {
    method: 'GET',
    path: '/api/users/1',
  },
  willRespondWith: {
    status: 200,
    body: {
      id: 1,
      name: like('Nazmul'), // type matching
      email: like('test@test.com'),
    }
  }
};
```

```
// Provider verifies the same contract
// If API changes break the contract, tests fail
```

*Interview tip: Contract tests are faster than E2E and catch integration issues between teams.*

## Q172

★★★★★

ADVANCED

### How do you set up CI test strategies?

#### Answer:

Layer your CI pipeline: 1) Lint + type check (fastest, catch syntax issues). 2) Unit tests (fast, high coverage). 3) Integration tests (medium, test interactions). 4) E2E tests (slow, test critical paths only). Run fast checks first — fail fast, save CI minutes.

```
# .github/workflows/test.yml
jobs:
  lint:
    steps:
      - run: npm run lint
      - run: npx tsc --noEmit

  unit:
    needs: lint
    steps:
      - run: npx jest --ci --coverage
      - uses: codecov/codecov-action@v4

  integration:
    needs: unit
    services:
      postgres: { image: postgres:15 }
      redis: { image: redis:7 }
    steps:
      - run: npm run test:integration

  e2e:
    needs: integration
    steps:
      - run: npx playwright test
```

*Interview tip: Run lint → unit → integration → E2E in order. Fail fast on cheap checks.*

## Q173

★★★★★

ADVANCED

### What is visual regression testing?

#### Answer:

Visual regression testing captures screenshots of components/pages and compares them against baselines pixel-by-pixel. Catches CSS bugs, layout shifts, and unintended visual changes that functional tests miss. Tools: Chromatic (Storybook), Percy, Playwright visual comparisons.

```
// Playwright visual comparison
test('homepage looks correct', async ({ page }) => {
  await page.goto('/');
  await expect(page).toHaveScreenshot('homepage.png', {
    maxDiffPixelRatio: 0.01, // 1% tolerance
  });
});

// Component-level with Storybook + Chromatic
// 1. Write stories for each component state
// 2. Chromatic captures screenshots in CI
```

```
// 3. Visual diff on each PR
// 4. Approve or reject changes

// Update baselines:
// npx playwright test --update-snapshots
```

*Interview tip: Visual tests catch the bugs that unit tests can't — like 'the button moved 2px left'.*

**Q174**

★★★★★

**ADVANCED**

## Explain load testing and performance testing.

### Answer:

Load testing simulates many concurrent users to find breaking points. Key metrics: response time (p50, p95, p99), throughput (RPS), error rate. Tools: k6 (modern, JS-based), Artillery, JMeter. Run load tests against staging, never production.

```
// k6 load test
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '1m', target: 100 }, // ramp up
    { duration: '3m', target: 100 }, // sustain
    { duration: '1m', target: 0 },    // ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'], // 95% under 500ms
    http_req_failed: ['rate<0.01'],   // <1% errors
  },
};

export default function () {
  const res = http.get('https://staging.myapp.com/api/products');
  check(res, {
    'status 200': (r) => r.status === 200,
    'fast response': (r) => r.timings.duration < 500,
  });
  sleep(1);
}
```

*Interview tip: k6 is developer-friendly (write tests in JS), runs locally or in the cloud, and has great reporting.*

**Q175**

★★★★★

**ADVANCED**

## Explain Mutation Testing and how it improves test suite quality.

### Answer:

Mutation testing evaluates the quality of your tests by injecting small bugs (mutations) into your production code and running your test suite. If your tests fail, the mutation is 'killed' (good). If your tests pass, the mutation 'survived' (bad), indicating a gap in test coverage or assertions. Tools like Stryker Mutator automate this.

```
// Original Code:
// if (age >= 18) { return true; }

// Mutated Code (Stryker injects mutation):
// if (age > 18) { return true; } // Changed >= to >

// If tests do not have a test case for age = 18,
// the mutation survives. You must add a test case:
expect(isAdult(18)).toBe(true);
```

*Interview tip: High code coverage doesn't mean high quality. Mutation testing verifies that your assertions actually assert the correct behaviors.*

Q176

★★★★★

INTERMEDIATE

## What is Behavior-Driven Development (BDD)?

### Answer:

TDD focuses on the developer's perspective (unit structure, APIs). BDD (Behavior-Driven Development) focuses on the user's perspective (behavior, business logic) using a common language that developers, QAs, and product managers can understand. It uses Given-When-Then format Gherkin syntax with tools like Cucumber.

```
# BDD Feature spec (Gherkin syntax)
Feature: User Login
  Scenario: Successful login with valid credentials
    Given the user is on the login page
    When they enter 'test@user.com' and 'password123'
    And click 'Submit'
    Then they should see the dashboard page
```

*Interview tip: BDD helps align business requirements directly with automated tests, reducing communication gaps between teams.*

Q177

★★★★★

ADVANCED

## How do you test for security vulnerabilities in CI/CD?

### Answer:

Implement three security testing layers: 1) SAST (Static Application Security Testing): scans code for patterns indicating security flaws (ESLint security, SonarQube). 2) Dependency Scanning (SCA): scans node\_modules for known CVEs (npm audit, Snyk). 3) DAST (Dynamic Application Security Testing): tests running app by simulating attacks (OWASP ZAP).

```
# GitHub Actions Security Scan workflow snippet
- name: Run npm audit
  run: npm audit --audit-level=high

- name: Run Snyk to check for vulnerabilities
  uses: snyk/actions/node@master
  env:
    SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
```

*Interview tip: Automating dependency scans in CI/CD prevents importing packages with critical security flaws into production.*

Q178

★★★★★

BASIC

## Explain the AAA (Arrange-Act-Assert) pattern in testing.

### Answer:

The AAA pattern is a standard structure for writing clean, readable, and maintainable unit tests. It divides a test into three distinct blocks: 1) Arrange: set up the target object, inputs, and mock behaviors. 2) Act: execute the function or component method under test. 3) Assert: verify that the output or state matches expectations.

```
test('calculates total with tax', () => {
  // Arrange
  const cart = new ShoppingCart();
  cart.addItem({ name: 'Book', price: 10 });
```

```
// Act
const total = cart.calculateTotal(0.1); // 10% tax

// Assert
expect(total).toBe(11);
});
```

*Interview tip: Keep the three blocks separated by blank lines and avoid mixing setup with assertions to keep tests readable.*

**Q179**

★★★★★

**INTERMEDIATE**

## How do you mock modules, dates, and timers in Jest or Vitest safely?

### Answer:

Use Jest/Vitest mocking utilities to isolate code. Mock modules using `jest.mock()`. Use `jest.useFakeTimers()` to control time, and reset mocks between tests using `afterEach(jest.clearAllMocks)` to prevent test interference. To mock dates, use `jest.setSystemTime()` so that new `Date()` is completely deterministic.

```
// Mocking Date and Timers
beforeEach(() => {
  jest.useFakeTimers();
  jest.setSystemTime(new Date('2026-06-12'));
});

afterEach(() => {
  jest.useRealTimers();
});

test('sends reminder after 24 hours', () => {
  triggerReminderTimer();
  jest.advanceTimersByTime(24 * 60 * 60 * 1000);
  expect(sendEmailMock).toHaveBeenCalled();
});
```

*Interview tip: Always call `useRealTimers()` after testing timers to avoid disrupting other asynchronous tests in your suite.*

**Q180**

★★★★★

**INTERMEDIATE**

## Explain the testing trophy model vs the testing pyramid.

### Answer:

The Testing Pyramid focuses heavily on unit tests, some integration, and very few E2E tests. The Testing Trophy (popularized by Kent C. Dodds) prioritizes Integration tests over unit tests. The trophy structure is: 1) Static (type checkers, linters), 2) Unit (isolated logic), 3) Integration (component rendering, API - widest part, best ROI), 4) End-to-End (critical user paths).

```
// Testing Pyramid vs Testing Trophy
// Pyramid: Unit (70%) -> Integration (20%) -> E2E (10%)
// Trophy: Static (Linter/TS) -> Unit (Core Logic) -> Integration (Component/API) -> E2E (Critical Paths)
// Integration tests provide the maximum ROI for web applications.
```

*Interview tip: Integration tests strike the best balance between execution speed and real-world confidence.*

## How do you handle flaky tests in E2E environments?

### Answer:

Flaky tests pass or fail randomly due to network delays, race conditions, or asynchronous renders. Fixes:

- 1) Playwright auto-waiting: don't use arbitrary sleeps (like `sleep(3000)`); instead, wait for specific DOM states or network responses (`waitForSelector`, `expect(locator).toBeVisible()`).
- 2) Isolation: ensure tests have independent, clean state.
- 3) Run retries in CI.

```
// Bad - arbitrary sleep causes flakiness
await page.click('#submit');
await page.waitForTimeout(3000);
expect(await page.textContent('#status')).toBe('Success');

// Good - locator auto-waits for condition
await page.click('#submit');
const status = page.locator('#status');
await expect(status).toHaveText('Success', { timeout: 5000 });
```

*Interview tip: Never use arbitrary hardcoded timeouts for waiting. Always wait for specific, verifiable DOM states or network endpoints.*

## Phase 8: API Design & GraphQL

20 Q&As

Q182

★★★★★

BASIC

**What are REST API design best practices?**

**Answer:**

REST rules: 1) Use nouns not verbs in URLs (/users not /getUsers). 2) Use HTTP methods correctly (GET=read, POST=create, PUT=replace, PATCH=update, DELETE=remove). 3) Use proper status codes. 4) Version your API. 5) Use consistent naming (plural nouns, kebab-case). 6) Support filtering, sorting, pagination.

```
// Good REST design
GET    /api/v1/users           // list users
GET    /api/v1/users/123      // get user 123
POST   /api/v1/users          // create user
PATCH /api/v1/users/123      // update user 123
DELETE /api/v1/users/123      // delete user 123

// Filtering & pagination
GET /api/v1/users?role=admin&sort=-createdAt&page=2&limit=20

// Nested resources
GET /api/v1/users/123/posts   // user's posts
POST /api/v1/users/123/posts  // create user's post
```

*Interview tip: APIs are contracts — breaking changes break clients. Version from day one.*

Q183

★★★★★

BASIC

**Explain HTTP status codes and when to use them.**

**Answer:**

5 families: 1xx (info), 2xx (success), 3xx (redirect), 4xx (client error), 5xx (server error). Key codes to memorize: 200 OK, 201 Created, 204 No Content, 301 Moved Permanently, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable, 429 Too Many Requests, 500 Server Error.

```
// Success
200 – OK (GET, PUT, PATCH)
201 – Created (POST)
204 – No Content (DELETE)

// Client errors
400 – Bad Request (invalid input)
401 – Unauthorized (not logged in)
403 – Forbidden (no permission)
404 – Not Found
409 – Conflict (duplicate email)
422 – Unprocessable Entity (validation failed)
429 – Too Many Requests (rate limited)

// Server errors
500 – Internal Server Error
502 – Bad Gateway
503 – Service Unavailable
```

*Interview tip: Use 401 for 'who are you?' and 403 for 'I know who you are, but you can't do this'.*

## What is the difference between GraphQL and REST?

### Answer:

REST: multiple endpoints, fixed data shapes, over/under-fetching. GraphQL: single endpoint, client specifies exactly what data it needs, no over-fetching. REST is simpler for CRUD. GraphQL shines when: multiple clients need different data shapes, or you need data from multiple resources in one request.

```
// REST – multiple requests, over-fetching
GET /api/users/1           // gets ALL user fields
GET /api/users/1/posts    // separate request
GET /api/users/1/friends  // another request

// GraphQL – single request, exact data
query {
  user(id: 1) {
    name
    email
    posts(limit: 5) {
      title
    }
    friends {
      name
    }
  }
}
```

*Interview tip: REST for simple CRUD APIs. GraphQL for complex, nested data with multiple consumers.*

## Explain API pagination: offset vs cursor.

### Answer:

Offset-based: `page=2&limit=20` (skip first 20, take next 20). Simple but slow on large datasets — DB must count all skipped rows. Cursor-based: `after=lastItemId&limit=20` — uses an indexed field to seek directly. Much faster at scale. Always use cursor for infinite scroll or large datasets.

```
// Offset pagination (simple, slow at scale)
GET /api/posts?page=3&limit=20
// SQL: SELECT * FROM posts LIMIT 20 OFFSET 40

// Cursor pagination (fast, scalable)
GET /api/posts?after=abc123&limit=20
// SQL: SELECT * FROM posts
//   WHERE created_at < cursor_timestamp
//   ORDER BY created_at DESC LIMIT 20

// Response includes cursor for next page
{
  data: [...],
  pagination: {
    hasMore: true,
    nextCursor: 'xyz789'
  }
}
```

*Interview tip: Cursor pagination is  $O(1)$  regardless of page number. Offset pagination is  $O(n)$  for page  $n$ .*

## How do you handle API error responses consistently?

### Answer:

Define a standard error response format and use it everywhere. Include: error code (machine-readable), message (human-readable), details (field-level errors for validation). Use consistent HTTP status codes. Never expose internal errors or stack traces to clients.

```
// Standard error format
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request validation failed",
    "details": [
      { "field": "email", "message": "Invalid email format" },
      { "field": "age", "message": "Must be >= 18" }
    ]
  }
}

// Error handler middleware
app.use((err, req, res, next) => {
  const status = err.statusCode || 500;
  res.status(status).json({
    error: {
      code: err.code || 'INTERNAL_ERROR',
      message: err.message,
      ...(process.env.NODE_ENV !== 'production' &&
        { stack: err.stack })
    }
  });
});
```

*Interview tip: Consistent error formats make client-side error handling predictable and debuggable.*

## What are API versioning strategies?

### Answer:

Three approaches: URL versioning (/api/v1/users — most common, explicit), header versioning (Accept: application/vnd.api+json;version=1 — cleaner URLs), query param (?version=1 — easy but messy). URL versioning is the most common and recommended — it's explicit and cacheable.

```
// URL versioning (recommended)
app.use('/api/v1', v1Router);
app.use('/api/v2', v2Router);

// Header versioning
app.use('/api/users', (req, res, next) => {
  const version = req.headers['api-version'] || '1';
  if (version === '2') return v2Controller(req, res);
  return v1Controller(req, res);
});

// Deprecation strategy
res.set('Deprecation', 'true');
res.set('Sunset', 'Sat, 01 Jan 2026 00:00:00 GMT');
res.set('Link', '</api/v2/users>; rel="successor-version"');
```

*Interview tip: When deprecating an API version, give clients at least 6 months notice with Sunset headers.*

## Explain GraphQL schema design.

### Answer:

Define types (what data exists), queries (how to read), mutations (how to write), and subscriptions (how to listen). Use strong typing, input types for mutations, enums for fixed values, and interfaces for shared fields. Design schema from the client's perspective, not the database.

```
type User {
  id: ID!
  name: String!
  email: String!
  posts(limit: Int = 10): [Post!]!
  role: Role!
}

enum Role { USER ADMIN MODERATOR }

type Post {
  id: ID!
  title: String!
  author: User!
  createdAt: DateTime!
}

input CreatePostInput {
  title: String!
  content: String!
}

type Query {
  user(id: ID!): User
  posts(cursor: String, limit: Int): PostConnection!
}

type Mutation {
  createPost(input: CreatePostInput!): Post!
  deletePost(id: ID!): Boolean!
}
```

*Interview tip: Design GraphQL schema from the client's needs, not the database structure.*

## What is the N+1 problem in GraphQL and how do you solve it?

### Answer:

When resolving a list of items with nested fields, each nested field triggers a separate DB query. 10 users → 10 separate queries for their posts = 11 total. Solution: DataLoader batches and deduplicates queries into a single batch call per tick of the event loop.

```
// Without DataLoader – N+1
// Query: { users { name posts { title } } }
// 1 query for users + N queries for posts

// With DataLoader – batched
const postLoader = new DataLoader(async (userIds) => {
  // One query for ALL user IDs
  const posts = await db.posts.findMany({
    where: { userId: { in: userIds } }
  });
  // Return posts grouped by userId
});
```

```

    return userIds.map(id =>
      posts.filter(p => p.userId === id)
    );
  });

// Resolver
User: {
  posts: (user) => postLoader.load(user.id)
}

```

*Interview tip: DataLoader is essential for any production GraphQL server — without it, performance is terrible.*

**Q190**

★★★★★

**INTERMEDIATE**

## What is tRPC and why is it popular?

### Answer:

tRPC gives you end-to-end type safety between client and server WITHOUT code generation or schemas. Define procedures on the server with Zod validation, and the client gets full TypeScript autocompletion and type checking. No API layer needed — types flow automatically.

```

// Server (tRPC router)
const appRouter = router({
  getUser: publicProcedure
    .input(z.object({ id: z.string() }))
    .query(async ({ input }) => {
      return db.user.findUnique({ where: { id: input.id } });
    }),

  createPost: protectedProcedure
    .input(z.object({
      title: z.string().min(1),
      content: z.string(),
    }))
    .mutation(async ({ input, ctx }) => {
      return db.post.create({
        data: { ...input, authorId: ctx.user.id }
      });
    }),
});

// Client — full autocompletion!
const user = await trpc.getUser.query({ id: '1' });
user.name; // TypeScript knows this exists!

```

*Interview tip: tRPC is perfect for full-stack TypeScript apps where you control both client and server.*

**Q191**

★★★★★

**INTERMEDIATE**

## What is rate limiting and how do you implement it for APIs?

### Answer:

Rate limiting controls how many requests a client can make in a time window. Prevents abuse, brute force attacks, and ensures fair usage. Common algorithms: Fixed Window (simple counter per time window), Sliding Window (smoother), Token Bucket (allows bursts). Return 429 with Retry-After header when limited.

```

// Token bucket algorithm (conceptual)
class TokenBucket {
  constructor(capacity, refillRate) {
    this.tokens = capacity;
    this.capacity = capacity;
  }
}

```

```

    this.refillRate = refillRate; // tokens per second
    this.lastRefill = Date.now();
  }
  consume() {
    this.refill();
    if (this.tokens > 0) {
      this.tokens--;
      return true; // allowed
    }
    return false; // rate limited
  }
}

```

```

// Response headers
res.set('X-RateLimit-Limit', '100');
res.set('X-RateLimit-Remaining', '42');
res.set('X-RateLimit-Reset', '1623456789');
res.set('Retry-After', '30'); // seconds

```

*Interview tip: Always include rate limit headers so clients can self-regulate.*

**Q192**

★★★★★

**INTERMEDIATE**

## Explain API documentation with OpenAPI/Swagger.

### Answer:

OpenAPI (formerly Swagger) is the standard for documenting REST APIs. Define endpoints, parameters, request/response schemas, and authentication in YAML/JSON. Tools auto-generate interactive docs, client SDKs, and mock servers. Good docs reduce support burden and onboarding time.

```

# openapi.yaml
openapi: 3.0.3
info:
  title: User API
  version: 1.0.0
paths:
  /api/users:
    get:
      summary: List users
      parameters:
        - name: page
          in: query
          schema: { type: integer, default: 1 }
      responses:
        '200':
          description: Success
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/User'
components:
  schemas:
    User:
      type: object
      properties:
        id: { type: integer }
        name: { type: string }

```

*Interview tip: Use swagger-jsdoc to generate OpenAPI from JSDoc comments in your code.*

## How do you design authentication for GraphQL?

### Answer:

Authentication in GraphQL happens in the context, not in individual resolvers. Parse the JWT/session in the context function and pass the user to all resolvers. Authorization happens per-resolver or per-field using directives or middleware. Never put auth logic in the schema itself.

```
// Context – parse auth once
const server = new ApolloServer({
  typeDefs,
  resolvers,
  context: async ({ req }) => {
    const token = req.headers.authorization?.split(' ')[1];
    const user = token ? verifyJWT(token) : null;
    return { user, db };
  },
});

// Resolver – check auth
const resolvers = {
  Mutation: {
    createPost: (_, args, ctx) => {
      if (!ctx.user) throw new AuthenticationError('Login required');
      return ctx.db.post.create({
        data: { ...args.input, authorId: ctx.user.id }
      });
    },
  },
};
```

*Interview tip: Use a directive (@auth) or middleware library to avoid repeating auth checks in every resolver.*

## Explain GraphQL subscriptions for real-time data.

### Answer:

Subscriptions let clients receive real-time updates via WebSocket. The client subscribes to an event, and the server pushes data whenever that event occurs. Used for: chat messages, live notifications, real-time dashboards. Subscriptions are the GraphQL equivalent of WebSocket events.

```
// Schema
type Subscription {
  messageAdded(roomId: ID!): Message!
  userTyping(roomId: ID!): User!
}

// Server resolver
Subscription: {
  messageAdded: {
    subscribe: (_, { roomId }) =>
      pubsub.asyncIterator(`ROOM_${roomId}`)
  }
}

// Publish from mutation
Mutation: {
  sendMessage: async (_, { roomId, text }, ctx) => {
    const msg = await db.messages.create(...);
    pubsub.publish(`ROOM_${roomId}`, {
      messageAdded: msg
    });
  }
};
```

```

    return msg;
  }
}

// Client
const SUB = gql`
  subscription OnMessage($roomId: ID!) {
    messageAdded(roomId: $roomId) { text author { name } }
  }
`;

```

*Interview tip: Use Redis PubSub adapter for subscriptions across multiple server instances.*

**Q195**

★★★★★

**ADVANCED**

## Explain webhook design patterns.

### Answer:

Webhooks are HTTP callbacks — your server calls the client's URL when an event occurs (reverse of polling). Design: register a URL, send POST with signed payload on events, expect 2xx acknowledgment, retry with exponential backoff on failure. Always verify webhook signatures.

```

// Webhook sender
async function sendWebhook(url, event, payload) {
  const body = JSON.stringify({ event, data: payload, ts: Date.now() });
  const signature = crypto
    .createHmac('sha256', secret)
    .update(body).digest('hex');

  await fetch(url, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'X-Webhook-Signature': signature,
    },
    body,
  });
}

// Webhook receiver (verification)
app.post('/webhooks/stripe', (req, res) => {
  const sig = req.headers['stripe-signature'];
  const event = stripe.webhooks.constructEvent(
    req.body, sig, endpointSecret
  );
  // Process event...
  res.json({ received: true });
});

```

*Interview tip: Always verify webhook signatures — unsigned webhooks can be spoofed by attackers.*

**Q196**

★★★★★

**ADVANCED**

## What is API gateway pattern and why is it important?

### Answer:

API gateway is the single entry point for all API requests. It handles cross-cutting concerns: authentication, rate limiting, request routing, response aggregation, protocol translation, and logging. Without it, every microservice must implement these concerns independently.

```

// API Gateway responsibilities:
// Client -> API Gateway -> Microservices
//

```

```
// 1. Authentication: verify JWT once
// 2. Rate limiting: per user/API key
// 3. Routing: /users -> user-service
// 4. Aggregation: combine responses
// 5. Caching: cache frequent reads
// 6. Transformation: REST -> gRPC
// 7. Monitoring: log all requests
```

```
// Popular gateways:
// - Kong (open source, plugin-based)
// - AWS API Gateway (managed)
// - Nginx (reverse proxy + gateway)
// - Express Gateway (Node.js native)
```

*Interview tip: API gateway simplifies clients — they talk to one endpoint instead of discovering many services.*

**Q197**

★★★★★

**ADVANCED**

**Explain the Idempotency pattern in API design and how to implement it.**

**Answer:**

An idempotent operation is one that produces the same result no matter how many times it is executed. GET, PUT, and DELETE are naturally idempotent. POST is not. To make POST (e.g., payment creation) idempotent: 1) Client sends a unique UUID in the Idempotency-Key header. 2) Server checks if this key exists in Redis. 3) If it exists, returns the cached response. 4) If not, acquires a lock, processes the request, stores the response in Redis with an expiration, and returns the response.

```
app.post('/payments', async (req, res) => {
  const key = req.headers['idempotency-key'];
  if (!key) return res.status(400).send('Missing Idempotency-Key');

  const cached = await redis.get(key);
  if (cached) return res.json(JSON.parse(cached));

  // Process payment...
  const response = await processPayment(req.body);
  await redis.set(key, JSON.stringify(response), 'EX', 86400); // 24h
  return res.json(response);
});
```

*Interview tip: Implementing idempotency keys is essential for critical write operations (like financial payments) to prevent duplicate transactions on network retries.*

**Q198**

★★★★★

**INTERMEDIATE**

**What is HATEOAS in REST API design and is it practical?**

**Answer:**

HATEOAS (Hypermedia As The Engine Of Application State) is a constraint of REST. It states that the API response should return not just data, but also links to actions/related endpoints the client can take next, making the API self-documenting. In practice, it is rarely fully implemented because it adds considerable response payload size and development complexity, and frontends usually hardcode navigation paths anyway.

```
// HATEOAS response example
{
  "id": 123,
  "name": "Nazmul",
  "_links": {
    "self": { "href": "/api/users/123" },
    "posts": { "href": "/api/users/123/posts" },
  },
}
```

```
    "delete": { "href": "/api/users/123", "method": "DELETE" }
  }
}
```

*Interview tip: While HATEOAS is a key constraint for level 3 REST maturity, most production systems use simpler REST APIs documented via OpenAPI/Swagger.*

**Q199**

★★★★★

**ADVANCED**

### Explain GraphQL schema stitching vs Apollo Federation.

**Answer:**

Both merge multiple sub-schemas into a single gateway schema. Schema Stitching: imperatively merges schemas at the gateway level using resolvers (requires gateway updates when sub-schemas change). Apollo Federation: declaratively composes schemas using subgraphs. Each subgraph defines its own schema and specifies how entities (types shared across services) are extended. The gateway automatically composes the schema dynamically.

```
# Subgraph A (User Service)
type User @key(fields: "id") { id: ID! name: String! }

# Subgraph B (Review Service)
extend type User @key(fields: "id") {
  id: ID! @external
  reviews: [Review!]!
}
```

*Interview tip: Apollo Federation is the modern enterprise standard for building a unified graph (Supergraph) across independent team microservices.*

**Q200**

★★★★★

**ADVANCED**

### How do you protect GraphQL APIs from abuse?

**Answer:**

Because GraphQL lets clients query arbitrary nested fields, malicious clients can crash servers. Protections: 1) Query Depth Limiting: inspect AST and reject queries exceeding a max depth. 2) Query Cost Analysis: assign costs to fields (scalar=1, list=5) and reject queries exceeding a max score. 3) Rate Limiting: limit based on total query complexity score per IP.

```
import depthLimit from 'graphql-depth-limit';
import { createComplexityLimitRule } from 'graphql-validation-complexity';

const server = new ApolloServer({
  typeDefs,
  resolvers,
  validationRules: [
    depthLimit(5),
    createComplexityLimitRule(1000)
  ]
});
```

*Interview tip: Always define query depth and complexity limits in production GraphQL servers to prevent Denial of Service (DoS) attacks.*

**Explain HTTP Caching headers and how they optimize API performance.****Answer:**

1) Cache-Control: directs caches (browsers, CDNs) how to cache (public, max-age=3600). 2) ETag (Entity Tag): a unique hash representing the response content. The client sends it back in the If-None-Match header. If content hasn't changed, the server returns 304 Not Modified with no body. 3) Last-Modified: date of last modification, returned with If-Modified-Since.

```
app.get('/api/resource', (req, res) => {
  const content = getResource();
  const etag = generateHash(content);

  if (req.headers['if-none-match'] === etag) {
    return res.status(304).end(); // Not Modified
  }

  res.set('ETag', etag);
  res.set('Cache-Control', 'public, max-age=60'); // cache 60s
  res.json(content);
});
```

*Interview tip: ETags save bandwidth and CPU by avoiding database queries and JSON serialization when data has not changed since the last request.*